The Art & Science of Prompting

A Complete Guide to Talking with Machines That Think

Veer Sukhadiya

Veer
Sukhadiya
Books

First Edition

Published by Veeer Sukhadiya Books, Gujarat, India

For every reader who suspected
there was a craft hiding inside the conversation.

Contents

Preface: Why This Book Exists

There was a moment, sometime in the last two years, when a quiet revolution changed the shape of human work. It did not arrive with sirens. It arrived as a blinking cursor inside a chat box, patient and waiting, and whoever learned to speak into that box well found that a great deal of the friction of modern labour — drafting, summarising, coding, analysing, explaining, brainstorming — began to dissolve.

The people who got the most out of this revolution were not the people with the most powerful computers, nor the people with the best degrees, nor even the people who had been building artificial intelligence for decades. They were the people who figured out how to ask. They were, in a word, prompters.

This book is for everyone who has ever stared at that blinking cursor and felt a small voice ask: am I doing this right? It is for the writer who suspects the AI could do more than produce grey paragraphs of competent mush. It is for the founder who needs the model to stop hallucinating client names. It is for the student who wants genuine understanding, not a glossy summary. It is for the developer who needs the code to work the first time, not the fifth. It is for the publisher, the researcher, the teacher, the designer, and the merely curious.

I wrote it because the existing material on prompting falls into two camps. On one side are long, jargon-heavy research papers written for machine-learning specialists. On the other are thin listicles of "10 prompts that will change your life" that age worse than milk. Between them lies a vast missing middle — a place where the craft can be laid out plainly, the reasoning behind each technique explained, and the habits of expert prompters taught the way any craft is taught: through principles, patterns, and practice.

I call what you will learn here "adaptive prompting" because the field changes month to month. Specific tricks go stale. Model versions improve, sometimes making an old technique

unnecessary, sometimes making a new one possible. What does not go stale is the underlying way of thinking. If you understand why a technique works, you will invent new ones on the day the old ones stop.

The book is organised in six parts. Part I lays the foundation — what prompting is, how models actually process language, and what every prompt is made of. Part II walks through the core techniques every prompter needs: zero-shot, few-shot, chain-of-thought, role-setting, and structured output. Part III takes you deeper, into the methods used by teams building production AI systems: self-consistency, tree-of-thoughts, ReAct, constitutional critique, and retrieval-augmented generation. Part IV is the heart of the book — the adaptive secrets that separate fluent prompters from beginners: meta-prompting, context budgeting, the iteration loop, tone control, and the art of taming hallucinations. Part V applies everything to five real-world arenas: writing, code, research, business, and education. Part VI looks forward — to agents, multimodality, ethics, and what comes next.

Diagrams appear throughout because some ideas in this field are spatial more than verbal; a picture of a token pipeline or a reasoning tree will sit in your memory long after a paragraph of description has faded. Appendices at the end collect the templates you will reach for most often and the pitfalls you will want to avoid.

A final note. This book will not make you good at prompting. Reading about tennis does not make you good at tennis. What it can do — what I hope it does — is give you the mental models and the vocabulary to practise deliberately. Prompting, more than most skills, rewards the practitioner who notices their own patterns, names what they see, and experiments with care. The practice is yours. The map is here.

— V.S., Ahmedabad, 2026

PART ONE

The Foundations

• • •

Chapter 1 — What Prompting Really Is

> *"The limits of my language mean the limits of my world."*
>
> — Ludwig Wittgenstein

If you type the word "hello" into a modern language model, it replies. If you type a five-thousand-word legal brief and ask for a one-paragraph summary, it replies. If you paste in a spreadsheet and ask for three charts and an executive briefing, it replies. The same tool does all of these things, and the only thing that changed between them is the words you put in. That is prompting.

But this definition, while technically correct, is dangerously thin. It suggests that prompting is mere typing, and that anyone who can type can prompt. The second claim is true in the same way anyone who can hold a pen can write a novel. A prompt is a piece of language used to steer an extraordinarily powerful pattern-matching machine toward a specific behaviour. The power comes from the machine. The steering is yours.

1.1 Prompting is programming with language

For fifty years, giving instructions to a computer meant translating human intent into the narrow, unforgiving grammar of a programming language. If you wanted a computer to sort a list, you had to say so in a syntax that permitted exactly one way to express the idea. A single missing semicolon produced an error. The computer did what you typed, not what you meant.

Prompting is different in kind. A language model does not require syntax. It requires intent expressed clearly enough that the intent can be recovered from your words. The same request can be phrased ten ways and often all ten will work. This feels, at first, like a relaxation of the old contract between humans and machines. In some ways it is. But it introduces a new kind of difficulty, one programmers have not had to face before: the difficulty of being clear about what you actually want.

I have watched seasoned engineers struggle with this. They are used to specifying every constraint up front because a compiler will punish them if they don't. Give them a language model and they tend to under-specify, assuming the model will "figure out" what they mean. The model usually does figure something out — just not necessarily what they had in mind. The result is a kind of disappointment that looks, on the surface, like the model failing. What actually failed was the specification. This is the first hard lesson of prompting: the model is not reading your mind. It is reading your words. The difference between a great prompt and a poor one is almost always the difference between thinking you have been clear and actually having been clear.

1.2 A model is a probability distribution over what comes next

Before we go further, you need a correct mental model of what is happening inside that chat window. A language model is, at heart, a very large function. You give it a sequence of text — the prompt. It computes, for every possible next word in its vocabulary, a probability that that word will come next. It picks one (usually the most likely, or a sample from the top of the distribution), appends it to the sequence, and repeats.

This is the entire trick. There is no reasoning engine. There is no memory of what you said last week. There is no hidden agent with its own agenda. There is just a function that, given a context, is very good at guessing what text plausibly continues it. Everything we call "intelligence" in modern AI emerges from this one operation run at scale over enormous amounts of training data.

Once you hold this picture in your head, a great many mysteries resolve. Why does adding "Let's think step by step" improve accuracy on maths problems? Because text that begins that way, in the training data, is usually followed by correct step-by-step reasoning. Why does asking the model to act as an expert cardiologist produce better medical answers than asking it flatly?

Because text attributed to experts is, on average, more careful. Why does a typo sometimes ruin a response? Because typos change the tokens, and tokens change the probabilities, and the probabilities are everything.

This is also why prompting is adaptive. The best prompt depends on what patterns exist in the training data the model saw. As models are retrained, those patterns shift. A phrasing that worked brilliantly in 2023 can become merely adequate in 2026 — not because the model got worse, but because it learned more graceful ways to be steered. The prompter's job is less to memorise tricks than to maintain the habit of testing what actually works now.

1.3 The three questions every prompt answers

Reduce every prompt, no matter how long, to what it tells the model. You will find it answers some combination of three questions.

The first is who. Who is doing the task? Who is the audience? The model adapts its register and depth to both, if you tell it. Leave "who" unspecified and it defaults to a generic, vaguely academic voice that is polite, hedged, and instantly recognisable. Specify "who" precisely — a witty columnist writing for bored commuters; a compliance officer writing for a board of directors — and the whole texture of the output shifts.

The second is what. What must the model do? Not what topic, not what subject, but what verb. Summarise? Rewrite? Compare? Critique? Generate? Translate? Classify? Vague verbs like "help me with" or "look at" produce vague results. Sharp verbs produce sharp results.

The third is how. How should the answer be shaped? A paragraph? A table? A bulleted list with three items exactly? JSON matching a specified schema? A poem in rhyming couplets? The model will default to prose if you don't say. It will default to medium length if you don't say. It will default to its trained register if you don't

say. Specifying "how" costs you ten seconds and saves you ten minutes of reformatting.

A prompt that handles all three questions explicitly is already better than ninety per cent of the prompts most people write. Almost everything else in this book is a refinement on top of that foundation.

1.4 What prompting is not

It will help to name a few misconceptions early. Prompting is not magic. There are no secret words that, once spoken, unlock hidden capabilities. Prompts that claim to — "jailbreaks", "DAN modes", the endless TikTok videos promising "the prompt that will 10x your productivity" — overwhelmingly fail to reproduce in practice and age out within weeks. Chasing them is a distraction from the real craft.

Prompting is not one-shot. The idea that you should write the perfect prompt in a single attempt and receive the perfect answer is a fantasy that will slow your learning. Real prompting is a dialogue, or at least a short series of drafts. Expect to iterate.

Prompting is not a replacement for thinking. A model will happily produce fluent nonsense when asked to reason outside its competence. The prompter who treats the model as an oracle will be embarrassed in public eventually. The prompter who treats the model as a very fast, very well-read, occasionally wrong colleague will get far more from it.

Prompting is not permanent. The techniques in Part IV are, as far as we can tell in 2026, durable. The specific phrasings in the Appendix are useful today and may be obsolete in a year. Build your knowledge on the durable layer and let the surface layer refresh itself.

Chapter 2 — How Language Models Think

> *"You don't need to know how an engine works to drive a car. You do need to know how one works to drive one fast."*
>
> — Anonymous race engineer

The goal of this chapter is not to make you a machine-learning researcher. It is to give you, in under half an hour of reading, a mental model accurate enough that your prompting intuitions will be correct most of the time. We will cover tokens, embeddings, attention, temperature, and the sampling loop that stitches them together. Every concept here will reappear, often, through the rest of the book.

2.1 Tokens: the atoms of the model

Computers do not see words. They see numbers. Before a model can do anything with your prompt, it must convert your text into a sequence of integers, each one standing for a piece of text called a token. A token is usually a short string — a whole word, a syllable, a prefix, a punctuation mark — chosen by a tokeniser trained to break text into efficient pieces.

Most common English words are one token. Uncommon words are two or three. Indian names, technical jargon, and emoji tend to cost more tokens than you would guess. A paragraph of ordinary prose runs at roughly one token per 0.75 words in English; in other languages and scripts the ratio is often worse. This matters because every model has a hard limit on the total number of tokens it can process at once — the context window — and because many commercial models are priced per token.

The practical implications are small but real. When you notice a prompt behaving strangely around a specific word, try rewording. Sometimes that word is being tokenised into an unexpected shape. When you need to pack more content into a limited window, switching to plainer English or shortening variable names in code

can save surprising amounts. When you want to estimate cost, remember that both your input and the model's output count, and that structured formats like JSON cost extra tokens for the punctuation.

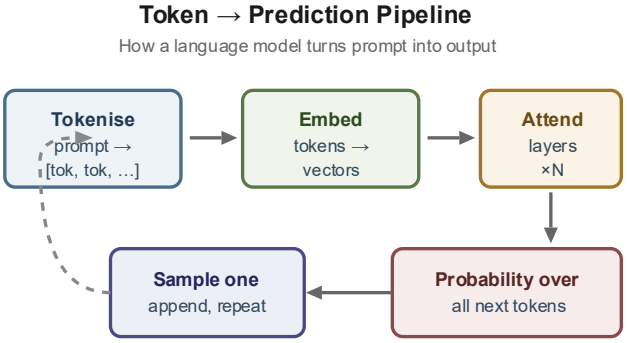


Figure 2.1 — From raw prompt to next token, the five stages every language model runs in sequence.

2.2 Embeddings: from tokens to meaning

Once your prompt is a list of token IDs, the model converts each ID into a vector of numbers — typically a few thousand numbers long. This vector is the token's embedding, and it represents the token's meaning in a geometric space the model learned during training. Tokens with related meanings end up at nearby points in this space. "King" and "queen" are close. "King" and "asparagus" are far.

This is not a metaphor. The geometry is real. You can take the vector for "king", subtract the vector for "man", add the vector for "woman", and arrive near the vector for "queen". The model did not learn this trick because anyone taught it to. It emerged because placing related concepts near each other turned out to be the most efficient way to predict the next token across billions of examples.

The reason this matters for prompting is subtle but important. When you use synonyms, the model often responds the same way because the synonyms sit in similar neighbourhoods. When you use a rare or technical term, you activate a narrower region of the space, which can either produce more precise output or, if the model knows the term poorly, produce errors. Good prompters develop an ear for which words are "central" in a field — the words a real expert would use — and prefer those. You will see examples throughout this book.

2.3 Attention: how context flows

The core operation of every modern language model is called attention. You can think of attention as follows: for each token the model is about to predict, it asks, of every token that came before, "how relevant is this one to what I'm trying to say next?" It computes a weight for each previous token, multiplies each token's embedding by its weight, sums them, and uses that sum to generate the next token.

Attention is what allows the model to hold a long document in working memory and still resolve a pronoun near the end back to a noun from the first paragraph. It is also what allows it to follow instructions: when you write "answer in French" at the top of your prompt, attention lets every subsequent generation step see that instruction and weight it heavily.

But attention is not free. It gets more expensive as the context gets longer — quadratically so, in the standard formulation. More importantly, it gets noisier. The more tokens the model has to choose between, the harder it is to pick out the few that matter. There is a well-documented phenomenon called "lost in the middle": facts placed in the middle of a very long context are often forgotten, while facts at the start or end are attended to reliably. This will shape many of the recommendations in Part IV.

2.4 Temperature and sampling

When the model has computed its probability distribution over the next token, it must pick one. The naïve choice is always to pick the most likely. This is called greedy decoding, and it produces repetitive, stilted output. Real generation uses sampling — drawing from the distribution with some randomness.

The knob that controls this randomness is called temperature. At temperature zero, the model is essentially greedy: it always picks the top choice, making the output deterministic and boring. At temperature one, it samples honestly from the distribution. At temperatures above one, the distribution is flattened: unlikely tokens become relatively more likely, and the output becomes creative, surprising, and often unhinged.

For factual tasks — coding, extraction, classification, structured output — lower the temperature toward zero. You want the most likely answer, consistently. For creative tasks — brainstorming, fiction, poetry, humour — raise it. The sweet spot is usually between 0.2 and 0.4 for work and between 0.7 and 1.0 for play. If your tool does not expose temperature, the interface has chosen a default for you; usually a middling value that compromises between both use cases.

> **Temperature ≠ creativity**
> High temperature does not make a model more intelligent or more original. It makes it less predictable. If your prompt is bad, a high temperature produces worse nonsense faster. Fix the prompt first; tune the temperature second.

2.5 The sampling loop

Putting it all together: when you send a prompt, the model tokenises it, converts the tokens to embeddings, runs them through dozens of layers of attention and feedforward networks, and produces a probability distribution for the next token. It samples one. It appends that token to the prompt. And it does the whole thing again — every single time, from scratch, for every token of the output.

This has several consequences that beginners miss. The first is that the model does not plan. It does not know, when it begins a sentence, how that sentence will end. It commits to each token one at a time, and if an early token leads the sentence toward nonsense, it will valiantly try to salvage the rest. This is why asking for a short answer sometimes produces better reasoning than asking for a long one: fewer tokens means fewer chances to drift.

The second is that whatever appears earlier in the context influences everything after. An example you give near the top will shape every token below it. A casual aside in the middle can nudge the tone of the whole response. Order matters far more than most people realise, and we will come back to this repeatedly.

The third is that the model cannot take back a token once it has been generated. Humans can revise a sentence they have written. The model cannot, unless you explicitly prompt it to. This is the mechanical reason why techniques that force the model to critique and rewrite its own output — which we will meet in Chapter 11 — lift quality so dramatically: they give the model a second chance.

Chapter 3 — The Anatomy of a Prompt

In Chapter 1 we said every prompt answers who, what, and how. In practice, well-engineered prompts consist of six distinct layers, each serving a specific purpose. Not every prompt needs every layer. But knowing the full anatomy lets you add or remove components deliberately, the way a composer adds or removes instruments.

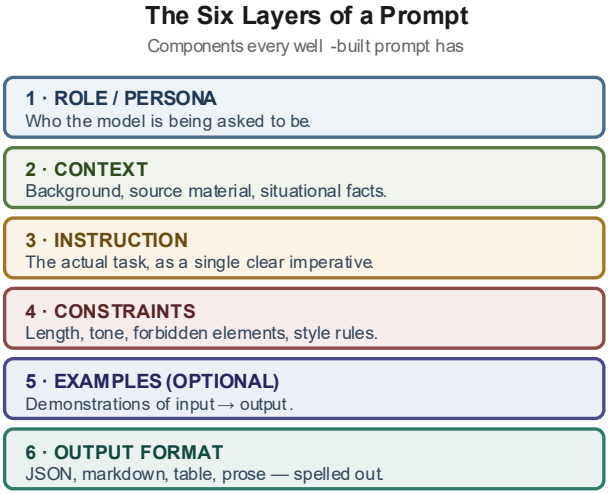


Figure 1.1 — The six layers of a prompt. Each one serves a distinct purpose; great prompts use them with intent, not habit.

3.1 Role

The role sets the voice. It is a single sentence, typically at the top of the prompt, that tells the model who it is: "You are a senior literary editor at a small press that specialises in speculative fiction", or "You are a meticulous accountant who double-checks every number". The role is powerful because it pulls the entire output toward a register associated with the named persona.

Roles work best when they are specific. "You are an expert" is weak because expertise in what? Of which kind? The phrase activates so much training data that it cancels out. "You are a veteran

emergency-room doctor in a busy urban hospital, used to explaining complex diagnoses to frightened patients in the calmest possible terms" is strong because it calls up a narrower, more consistent region of the model's space. The output will carry that persona's patience and precision.

Beware of over-claiming. Asking the model to be "the world's leading expert on quantum field theory" does not summon one from thin air. It still has the same underlying knowledge. What it does is select a more confident, more technical voice. If the question exceeds the model's actual knowledge, that confident voice will produce confidently-wrong answers. The role affects style more than it affects capability.

3.2 Context

Context is the information the model needs about the specific situation that it could not know by default. Source material to summarise. A company's style guide. The transcript of a customer conversation. The text of a law. Every time you find yourself wishing the model "just understood" something, the fix is to add that something as context.

The discipline here is to give context rather than describe it. "Our company values concise communication" is much weaker than pasting three real examples of the company's preferred writing. Models learn from examples more efficiently than from descriptions — a phenomenon sometimes called "show, don't tell" that we will return to in Chapter 4.

Context is also where most prompts get too long. Every token of context is a token the model's attention must weigh, and beyond a certain point adding more context causes performance to plateau or decline. We will look at context budgeting in detail in Chapter 14.

3.3 Instruction

The instruction is the imperative — the actual task, stated as clearly as you can manage. This is the one layer no prompt can omit. It should be one sentence if at all possible, placed after the context, and written in the active voice. Most weak prompts fail here first: the "instruction" is actually a vague wish. "Help me with this email" is a wish. "Rewrite this email to sound more confident while keeping its length under 80 words" is an instruction.

Good instructions start with a verb. Summarise. Classify. Extract. Rewrite. Translate. Critique. Generate. Every one of these maps to a different capability the model has been trained on separately, and picking the right verb orients the model toward the right capability. When you find yourself writing "look at", "go through", or "work on", pause. Those are not verbs a model can execute. Replace them.

3.4 Constraints

Constraints are the negative space of the instruction — the things the model should not do, the limits it should respect, the forms it should avoid. "Keep the response under 200 words." "Do not speculate; mark anything uncertain as UNKNOWN." "Do not use any adjective more than once." "Never apologise."

Two things surprise new prompters about constraints. First, they work better than you expect. A model asked to avoid apologies genuinely avoids them. Second, they work worse than you hope if framed as positives. "Be concise" is weaker than "Keep the entire response to three sentences or fewer". Concrete, checkable constraints outperform abstract ones because the model can, at every token, ask itself whether it is still within them.

Constraints are also where you prevent the failure modes you have seen before. Every time a prompt fails in a specific way — too long, too formal, invented citations, refusing unnecessarily — you can add a constraint that directly addresses that failure. Over time, your prompts accumulate a small library of constraint-shaped scar tissue, and they get more robust.

3.5 Examples

Examples — demonstrations of input paired with the desired output — are the single most powerful technique in prompt engineering. They let you show the model what you want instead of describing it, which is almost always more efficient and more effective. A prompt with two good examples often beats a prompt with five hundred words of explanation.

This is important enough to have its own chapter (Chapter 4), so we will not belabour it here. The point for now is structural: examples sit between the constraints and the instruction, in their own clearly-marked block, formatted consistently so the model can see the input-output pattern at a glance.

3.6 Output format

The last layer tells the model what the output should physically look like. A paragraph? A table with specified columns? Markdown? JSON that matches a particular schema? An XML structure? A single line with no punctuation? Spelling this out matters because the model's default format is rarely the one you want to do anything useful with downstream.

When you need structured data — and increasingly, for any serious automation you will — specify the schema precisely. Not "return as JSON" but "return as a JSON object with the following keys, no other keys, and no prose outside the JSON: {name: string, age: integer, tags: array of strings}". The more precise the format spec, the more reliably the model will follow it.

When you need prose, specify the structure anyway. "Start with a one-sentence summary, then three bullet points of supporting evidence, then a closing sentence that names the most important implication." The model will follow such a structure with high fidelity and the resulting output is more scannable than whatever shape it would have invented on its own.

The Six-Layer Drill

Before sending any important prompt, scan it once with the six layers in mind. Is each one present and adequate? If one is missing, ask yourself whether that was a deliberate choice or an oversight. Most first drafts are missing at least two layers. Adding them takes a minute and typically moves the quality needle further than any other single improvement.

PART TWO

The Core Techniques

• • •

Chapter 4 — Zero-Shot, One-Shot, and Few-Shot Prompting

Before we had language models of any real sophistication, teaching a computer a new task required thousands of labelled examples and a training run that took hours or days. Modern language models have quietly abolished this bottleneck. You can now teach a model a new task in the time it takes to write an email — sometimes with zero examples, sometimes with one, sometimes with a handful. The technique is called shot prompting, and it is the workhorse of applied AI.

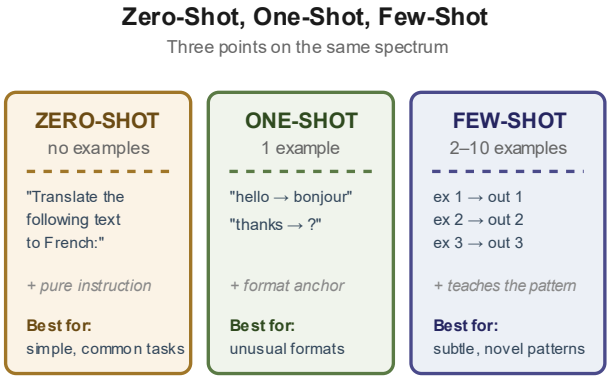


Figure 4.1 — The shot spectrum. Zero, one, and few-shot prompting trade example count against token cost and task difficulty.

4.1 Zero-shot prompting

Zero-shot prompting means asking the model to perform a task without giving it any examples of that task being performed. You describe the task, the model does it. For any well-known task — writing a summary, translating a sentence, classifying sentiment as positive or negative — zero-shot is the right starting point. It is cheap, fast, and often sufficient.

A good zero-shot prompt leans on the model's vast training data. If the task is common in that data, the model has effectively seen

it a million times and will perform it competently from a clear instruction alone. "Translate the following paragraph into Hindi, preserving the formal register: [paragraph]" needs no examples. Everyone in the world who has ever translated a paragraph did so in the training data.

Zero-shot starts to struggle when the task is unusual, when the format is bespoke, or when the labels you want to use are not the standard ones. "Classify this review using our internal taxonomy of twelve customer-experience themes" is not a task the training data has ever seen, because your internal taxonomy is yours. Zero-shot will still return twelve-ish themes, but they will be the model's guess at what your taxonomy might be, which is probably not what it actually is.

The test for whether zero-shot will suffice is simple: if a bright new hire could do this task on their first day from the instruction alone, the model probably can too. If the new hire would need examples, so does the model.

4.2 One-shot prompting

One-shot prompting means giving the model exactly one example of the task before asking it to do the next one. This is often enough to pin down format, tone, or the specific labels you want used. It is surprisingly effective and surprisingly underused.

The classic example is classification with custom labels. Zero-shot, if you ask the model to classify a piece of feedback as "bug", "feature request", or "praise", it will often return words like "complaint" or "suggestion" that sound like your categories but are not your categories. One-shot fixes this: show it one feedback item and the exact label it should have, and from then on it will use your words.

One-shot is also the cheapest way to control output format. If you want every answer to be exactly three bullet points starting with an action verb, give the model one such answer. The structure will propagate. If you want it to cite sources a particular way, show it

one citation in that style. The power of a single example is that it conveys information — about format, tone, length, precision — that prose descriptions struggle to capture.

4.3 Few-shot prompting

Few-shot — typically between two and ten examples — is the right move when the task has edge cases the model needs to see, when the taxonomy is genuinely complex, or when the style you want is subtle. With more examples, you can show the model how to handle tricky inputs, how to resolve ambiguities, and how to handle cases where the obvious answer is wrong.

The craft of few-shot prompting is in choosing the examples. The principle most people get wrong: your examples should not be the easy cases. The easy cases, by definition, need the least help from examples. Your examples should be the tricky ones — the ones where a zero-shot approach would fail, each illustrating a different failure mode. This way the model sees, in advance, every kind of mistake it would otherwise make.

A second principle: vary the examples. If all five of your examples are positive-sentiment reviews, the model will bias toward calling things positive. Include a balanced mix. This applies beyond sentiment. If you are showing examples of business emails, include polite ones and firm ones, short ones and long ones, replies and cold outreach. Variety in examples produces robustness in behaviour.

A third principle: show the reasoning, not just the answer. When the task has any complexity, include in each example not just the input and the final output but the intermediate work — the extracted entities, the applied rule, the chosen label with a one-line justification. This turns a few-shot prompt into a few-shot chain-of-thought prompt, a hybrid we will see again in the next chapter.

4.4 How many examples are enough?

There is no single correct number. But a rough curve holds across many tasks: the first example produces a large jump in quality. The second produces a smaller jump. By about five, you are usually near the plateau. Adding more examples beyond that starts costing tokens for diminishing returns, and at some point the sheer length of the prompt begins to hurt performance through the lost-in-the-middle effect we met in Chapter 2.

If you find yourself needing twenty examples to get acceptable quality, the task is probably too complex for few-shot alone. At that point consider one of three moves: (1) break the task into simpler sub-tasks, each with its own focused few-shot prompt; (2) use a more capable model; or (3) if the task is large enough to justify it, consider fine-tuning — actually retraining a version of the model on your examples. Fine-tuning is outside the scope of this book but becomes worth considering when you have more than a few hundred clean examples and you need the task performed identically at scale.

4.5 The format of examples

Examples should be formatted consistently, with clear delimiters separating input from output and one example from the next. The specific format matters less than consistency. A common pattern:

> Input: <the input> Output: <the output> Input: <the input> Output: <the output> Input: <the actual question> Output:

That last line — "Output:" with no content — is important. It primes the model to complete the pattern by producing the output for the actual question. Without it, the model sometimes rephrases the question or adds commentary before answering. Small things like this, invisible to humans, make large differences to model behaviour. This is an example of the general principle that the model is a pattern-completion machine: give it a clear pattern and it will complete it faithfully.

The Example Budget

When you add an example to a prompt, ask: what specific failure mode does this example prevent? If you cannot name one, the example is probably not earning its tokens. Every example should exist for a reason. Run the prompt without it occasionally to verify the reason is still real.

Chapter 5 — Chain-of-Thought Reasoning

In 2022, a paper from Google researchers quietly upended what people thought was possible with prompting. They had noticed that when language models were asked to solve multi-step problems, they often got them wrong — but when they were asked to show their working first, and then give the answer, their accuracy jumped dramatically. The technique was called chain-of-thought prompting, and it remains one of the most important ideas in the field.

5.1 Why it works

Recall from Chapter 2 that a language model generates one token at a time, and that each token is influenced by every token that came before it. When you ask the model for an answer directly, it must commit to that answer in a handful of tokens, with no room to work through the problem. When you ask it to think step by step first, each step of the thought becomes part of the context the final answer is conditioned on. The model is, in effect, scratch-padding in public.

This is not just an analogy. Empirically, on problems that require multi-step arithmetic, logical deduction, or careful analysis, chain-of-thought reasoning produces measurable improvements — sometimes doubling or tripling accuracy. The improvements grow with model size. The largest models benefit from chain-of-thought the most, which suggests the capability is latent in them and is simply being unlocked by the prompt.

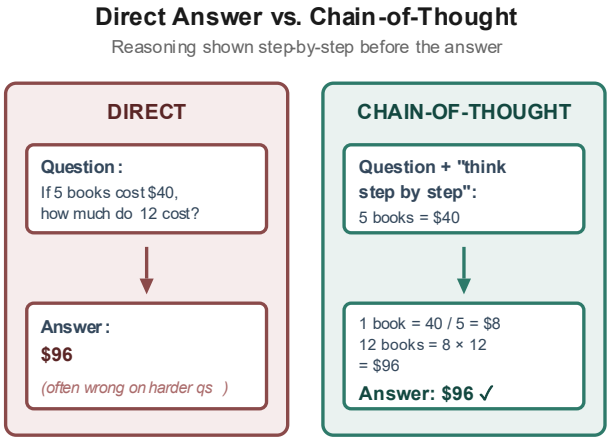


Figure 5.1 — Direct answer vs chain-of-thought on the same question. The intermediate reasoning gives the model room to correct itself mid-generation.

5.2 The magic sentence

The simplest chain-of-thought prompt is a single sentence appended to any question: "Let's think step by step." In the research literature this trick got its own name — "zero-shot chain-of-thought" — because it works without any examples. For routine multi-step problems, it is often enough.

Other phrasings work too. "Let's work through this carefully." "First, let me understand what's being asked." "Walk me through the reasoning, then give the final answer." The specific phrasing does not matter much; what matters is that you trigger the model's mode of producing intermediate work. Any phrasing that sounds like the opening of a careful explanation will do.

5.3 Structured chain-of-thought

For complex tasks, the magic sentence is blunt. A more controlled version spells out the steps you want the model to take. "Before answering, (1) identify the key entities in the question, (2) state what each one means in the context of the problem, (3) list the

constraints that apply, (4) work through the logic, and then (5) give your final answer." This kind of structured chain-of-thought is especially useful for analytical tasks where you care about the kind of reasoning, not just the conclusion.

The pattern generalises. Any task where you can articulate a procedure can be prompted as a sequence of steps. Writing a business proposal? Ask the model to (1) restate the client's problem, (2) list their constraints, (3) propose three approaches, (4) evaluate each against the constraints, (5) recommend one with a justification. The structured steps force the model to do the analytical work explicitly, which usually produces a better recommendation than asking for one directly.

5.4 When chain-of-thought hurts

Chain-of-thought is not free. It produces longer responses, which cost more tokens and take more time. For simple tasks where the answer is obvious from the question, forcing the model to reason through it is worse than useless: it creates opportunities for the model to overthink and arrive at a stranger answer than the direct one. For classifying an email as spam or not spam, you do not want five sentences of reasoning. You want a one-word answer.

A useful heuristic: chain-of-thought helps when the human version of the task would benefit from a scratch pad. If you, a human, would need to write something down to get the answer right, the model probably does too. If you can answer the question in your head without thinking, the model probably can too, and asking it to reason will just add noise.

5.5 Few-shot chain-of-thought

The most powerful variant combines the two techniques. You provide a few examples of the task being performed, and in each example you include not just the input and the final answer but the chain of reasoning in between. The model learns, from your

examples, the specific style of reasoning you want — what to check first, what to ignore, how to weigh competing concerns.

This is especially valuable for domain-specific reasoning. An example might show how a tax advisor reasons through a capital-gains question: first confirming the asset class, then checking the holding period, then applying the relevant rate, then considering exemptions. The model, given two or three such examples, will then reason about a new question in the same pattern. This is how production systems get reliable domain behaviour without fine-tuning.

Chapter 6 — The Power of Roles and Personas

If a prompt is a piece of language used to steer a model, then the role you assign is the steering wheel's centre position. A well-chosen persona changes the register, the depth, the confidence, the vocabulary, and the priorities of the output in ways that would take pages of explicit constraints to reproduce. A badly chosen persona does nothing, or worse, pulls the model toward fluent but unreliable output.

6.1 What a role actually does

A role is not a costume the model puts on. The model is not pretending to be a cardiologist when you tell it to act as one. What happens, mechanically, is that the text of the role description shifts the attention weights throughout the generation. Tokens associated with cardiology-adjacent concepts become slightly more likely. Tokens associated with the register used by experts become slightly more likely. The cumulative effect is output that sounds and reads more like what a cardiologist would actually produce.

This is why roles work best when they are specific. "You are a doctor" activates a large, heterogeneous region of the model's space — it includes paediatricians, GPs, surgeons, medical students, TV doctors, and every other doctor-shaped use of language in the training data. "You are a cardiac electrophysiologist with twenty years of experience running a busy clinic in Mumbai" activates a much narrower, more consistent region. The output will carry the specific texture of that narrower region.

6.2 The persona toolkit

Every persona has several dimensions you can tune.

Expertise level: novice, practitioner, expert, or leading authority. Novices explain things carefully; experts assume background

knowledge and cut straight to nuance. If you need patient tutorial prose, specify a novice-friendly expert. If you need dense technical analysis, specify an expert talking to peers.

Temperament: patient, blunt, cautious, bold, sceptical, encouraging. A "sceptical senior editor" produces different feedback on a draft than a "supportive writing coach", even if their underlying knowledge is similar. Match the temperament to what the task actually needs.

Audience: who the persona is talking to. A surgeon explaining a procedure to a colleague produces different text from a surgeon explaining the same procedure to a terrified patient. The audience shapes the vocabulary, the hedging, and the emotional texture.

Goal: what the persona is trying to accomplish. A journalist trying to inform differs from a journalist trying to persuade differs from a journalist trying to entertain. Stating the goal prevents the model from quietly defaulting to its trained baseline, which tends toward "inform evenly".

6.3 The credential illusion

A common mistake: assuming that loftier credentials produce better answers. The model knows what a Nobel laureate sounds like — confident, dense, allusive — but it does not suddenly know more chemistry when you tell it is one. The role affects style more than capability. If the underlying question exceeds the model's actual knowledge, a grander persona will produce a more confident wrong answer, which is worse than a humbler wrong answer because it is harder to catch.

Use roles honestly. Assign the level of expertise the task actually requires. For a complicated medical question, "You are an experienced internist explaining this to a medical student — when uncertain, say so explicitly" is a better role than "You are the world's greatest diagnostician". The first produces careful output that admits its limits. The second produces confident output that may invent its limits.

6.4 Multi-role prompts

Some tasks are best approached from multiple angles. You can instruct the model to reason from several roles in sequence. "First, analyse this marketing copy as a brand strategist would: does the positioning make sense? Then, analyse it as a copy editor would: is the prose clean? Then, analyse it as a first-time reader would: is anything confusing? Give me your notes from each perspective separately." This triples the useful feedback without tripling the work.

A more advanced version: have the model simulate a panel. "Three experts are reviewing this proposal — a product manager, a design lead, and a finance director. Write their feedback as a short transcript, with each person speaking in their own voice and focusing on what matters to them." The model produces meaningfully different critiques from each seat, and you get a more rounded picture of the proposal's strengths and weaknesses.

The Honest-Expert Default

For serious work, a reliable default role is something like: "You are an experienced practitioner in [field]. You are careful about the limits of your own knowledge. When you are uncertain, you say so, briefly and without hedging into uselessness. You never invent citations, references, or statistics. If you do not know something, you say 'I don't know' and explain what would be needed to find out." This single role prevents a large fraction of hallucination issues before they start.

Chapter 7 — Structured Outputs and Format Control

A response you cannot use is not a response. Much of the frustration people feel with language models comes not from the content of the output but from its shape: a wall of prose when you wanted a table, an essay when you wanted JSON, five paragraphs when you wanted three bullet points. This entire category of problem has a straightforward solution: specify the format. Explicitly. Every time.

7.1 Why default formats are rarely useful

A model with no format instruction defaults to the format it saw most often in its training data for questions of that general shape. For an essay-like question, that is paragraphs of prose. For a technical question, it tends to be a mix of paragraphs and bullet points. For a code question, it is code blocks with surrounding commentary. These defaults are fine for casual conversation and inadequate for almost any productive use.

If you plan to read the output once and act on it yourself, the shape matters less. If you plan to paste it into another document, process it with a script, show it to someone else, or compare it with other outputs, the shape matters enormously. Spending ten seconds to specify the format can save you ten minutes of reshaping the response by hand — or, in automated pipelines, the time it takes to debug why the downstream system is choking on the model's output.

7.2 Plain prose with explicit structure

Even when you want prose, telling the model what the prose should contain and in what order produces far better output than leaving it to choose. A template like "Write a three-paragraph analysis. Paragraph 1: state the core issue in plain terms. Paragraph 2: explore two possible interpretations and their implications. Paragraph 3: recommend a course of action with

specific next steps" will produce a response you can read at a glance and that consistently covers the ground you asked it to.

The same approach works for longer content. A book chapter might be structured as "Opening hook paragraph, background paragraph, three examples with commentary, synthesis paragraph, closing line". The model, given this structure, will follow it. Without it, the shape of each chapter will drift.

7.3 Markdown

For any output you plan to read in a formatted viewer — a report, a memo, a documentation page — Markdown is the lingua franca. Models produce it fluently and it renders cleanly in most tools. Specify it explicitly: "Return the analysis in Markdown. Use H2 headings for each major section, bulleted lists for enumerations, and fenced code blocks for any code or structured data."

Markdown is also useful when you want semi-structured output that a human will scan but that also needs some programmatic processing. Tables in Markdown are trivial to parse. Headings can be used as anchors. Code blocks can be extracted cleanly. It is a middle point between free prose and strict JSON that often hits the sweet spot for real work.

7.4 JSON and other strict schemas

For anything you will process with code, JSON is the right answer. Specify the exact schema, including keys, types, whether each key is required, and any constraints on values. Ask the model to return only the JSON, with no prose wrapping. Show one example of a valid response if the schema is non-trivial.

> Return a JSON object matching this schema exactly: { "theme": string, // one of: "growth", "risk", "cost", "other" "confidence": number, // between 0 and 1 "supporting_quote": string, // verbatim text from the input "rationale": string // one sentence } Return ONLY the JSON object. No markdown fences, no prose.

Several model providers now offer "structured output" modes that guarantee valid JSON against a supplied schema. When those

modes exist, use them — they eliminate the remaining cases where the model would otherwise occasionally produce invalid JSON under pressure. When they do not, specifying the schema in the prompt gets you most of the way there.

7.5 Tables, lists, and the humble comma-separated value

Tables are often the clearest format when you have several items with parallel attributes — a shortlist of candidates, a comparison of products, a catalogue of errors. Ask for a Markdown table with specified columns. For tabular data that will be pasted into a spreadsheet, ask for comma-separated values or tab-separated values instead, and warn the model that commas inside fields must be escaped.

Lists are the clearest format when you have a set of items with no particular attributes — alternatives to consider, steps to take, questions to ask. Specify whether you want a numbered list (for ordered or countable items) or a bulleted list (for unordered ones), and specify the maximum number of items. "Give me five" is almost always better than "give me a list", because otherwise the model will give you fifteen and the marginal items will be filler.

7.6 The format-first prompt

For any prompt you intend to run more than once, consider putting the format specification at the very beginning. "I need output in this exact shape: [schema]. Now here is the task: [task]." This format-first framing makes the model's goal explicit before it even reads the task, and in my experience produces higher compliance rates than putting the format spec at the end.

When you are building a system that will call the model many times with different inputs, the format spec should live in a stable template, with only the input varying. You write and iterate on the template once, and the behaviour of the whole system becomes

predictable. We will return to this discipline in Part IV, because it is the foundation of every reliable AI workflow.

PART THREE

Advanced Prompt Engineering

• • •

Chapter 8 — Self-Consistency and Reasoning Ensembles

If chain-of-thought prompting is one model producing one line of reasoning, self-consistency prompting is one model producing many and letting the majority rule. The idea is borrowed from the oldest and most reliable technique in applied statistics: when a single estimate is noisy, take several and combine them. Language models, despite looking deterministic, are noisy in exactly the right way for this to work.

8.1 The mechanism

To run self-consistency, you submit the same chain-of-thought prompt multiple times at a non-zero temperature so the model produces genuinely different reasoning paths. You collect the final answers. You return the answer that was reached most often. On hard reasoning problems, this vote-of-the-reasonings beats a single chain of thought, sometimes by wide margins.

The reason it works is that a single chain of thought can wander into a subtle error and confidently deliver the wrong answer. Ten chains of thought can also wander — but they tend to wander into different errors. The correct answer, meanwhile, tends to be reached through several different valid reasoning paths. When you pool the answers and take the plurality, the correct answer wins more often than any individual wrong answer.

8.2 When to use it

Self-consistency is expensive — you are paying for several runs instead of one — and it only helps when the task has a clear, comparable final answer. Use it when the question is hard, when a wrong answer would be costly, and when the output can be reduced to a single value or label that you can take a majority vote on. Maths problems, classification tasks, multiple-choice reasoning, code generation that has to pass a specific test — all good fits.

Do not use it for open-ended generation. You cannot take a majority vote on which version of a poem is best. For creative tasks, sampling multiple outputs and picking your favourite is a different and simpler technique: you are the judge, not the consensus.

8.3 Ensemble variations

Beyond self-consistency, there are other ways to combine multiple model runs. You can have the model produce a first answer and a second, different attempt, then compare them and choose the stronger. You can have it produce an answer, then have a second call from the same model critique it with a fresh context, then a third call synthesise the critique and the answer into a final response. Each variation adds cost but can lift quality on tasks where the model is operating near its limits.

The unifying principle: treat one model call as a sample, not an oracle. A single sample is often right. When being right matters, take more samples. This mindset shift — from model-as-answer-machine to model-as-noisy-but-useful-estimator — is, in my experience, one of the biggest jumps a prompter can make in how seriously they take the outputs they get.

Chapter 9 — Tree of Thoughts and Multi-Path Reasoning

Chain-of-thought is a straight line. Self-consistency is several parallel lines. Tree of Thoughts — the name given to a technique first described in a 2023 paper by researchers at Princeton and Google DeepMind — is a branching tree. At each reasoning step, the model considers multiple next moves, scores them, keeps the promising ones, and expands them further. It is the closest thing the field has to chess-style game-tree search, and for certain problems it is dramatically more capable than the simpler techniques.

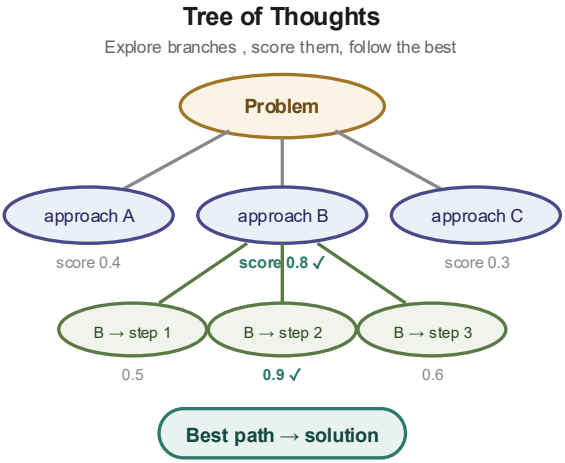


Figure 9.1 — Tree of Thoughts. The model generates several candidate approaches at each step, scores them, prunes the weak ones, and continues from the strongest.

9.1 The structure

The mechanics are straightforward in principle, even if implementations vary in sophistication. For a given problem, the model is asked to generate several distinct first moves — different framings, different decomposition strategies, different starting assumptions. Each first move is then scored, either by the same

model critiquing its own proposals or by a separate evaluator. The weakest are pruned. For the strongest, the model generates several second moves. And so on, until the tree reaches a depth where a concrete answer can be produced.

In practice, tree-of-thoughts prompts often use a handful of depth levels and a handful of branches per level — a tree of maybe a dozen total nodes. This is already much more reasoning than a straight chain, and for problems where the first move matters enormously, it works noticeably better.

9.2 Where it helps

Tree of Thoughts is designed for problems where many approaches are plausible but only a few will work, and you cannot tell which without exploring them part of the way. Puzzle-solving is the archetype. But the same structure applies to planning problems — choosing an architecture for a piece of software, designing an experiment, outlining a marketing strategy. In each case, the choice of approach at the top dominates the quality of the eventual answer, and committing too early produces suboptimal results.

For most everyday tasks, tree of thoughts is overkill. Chain-of-thought is sufficient for anything where the reasoning is roughly linear. The moment you notice yourself wishing the model would compare alternatives rather than commit to the first one that occurred to it, you have found a place where tree-of-thoughts-style prompting will help.

9.3 A practical tree-of-thoughts prompt

You do not need a research framework to get the benefits. A serviceable tree-of-thoughts prompt looks like this:

> For the problem below, do the following: 1. Generate THREE distinct approaches to the problem. Label them A, B, C. For each, write 2-3 sentences describing the approach and its key risk. 2. Score each approach from 1 to 10 on (a) likely correctness, (b) simplicity, (c) robustness to edge cases. Explain the scores briefly. 3. Choose the single best approach.

> 4. Execute that approach and produce the final answer. Problem: [the problem]

What you have done, structurally, is forced the model to do what a careful expert would do: consider the alternatives before committing. This alone, without any fancy framework, produces outputs that are noticeably more considered than a direct request for an answer. For work where getting the framing right matters more than the specific execution, this is one of the highest-leverage prompt patterns in existence.

Chapter 10 — ReAct: Reason + Act

Everything we have covered so far has concerned a model producing text from its own internal knowledge. But the world contains far more information than any model has been trained on, and some of what the model knows is out of date by the time it is asked. What if, in the middle of its reasoning, the model could call on external tools — a search engine, a database, a calculator, an API — and bring the results back into its own context? That capability, usually combined with chain-of-thought-style reasoning about what to do next, is called ReAct. It is the foundation of every AI agent in production today.

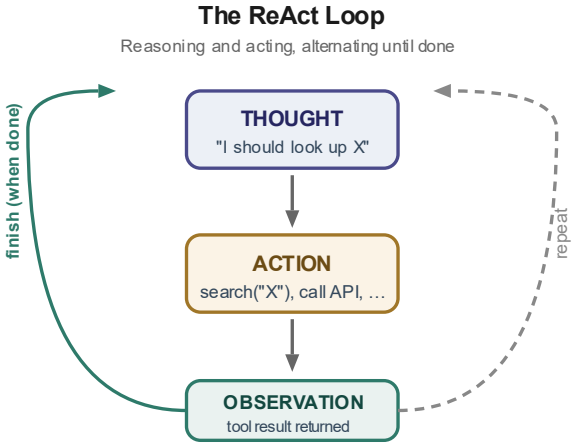


Figure 10.1 — The ReAct loop. Every modern AI agent runs on some variant of this three-step cycle.

10.1 The loop

ReAct alternates between three kinds of step. A Thought, in which the model reasons in language about what to do next. An Action, in which the model calls some external tool with specified arguments. An Observation, in which the tool returns a result that becomes part of the model's context. Then back to Thought, until the problem is solved or the model decides it has enough to produce a final answer.

The key insight is that each tool call gives the model information it did not have, in a form it can reason about. If the question is "what was our revenue last quarter?", the model cannot know. But it can notice that it cannot know, decide to query a database, read the result, and then answer. If the question is "compare the approach in these three PDFs", the model cannot read the PDFs by default — but it can ask a tool to extract their text, read it, and compare. The model's own reasoning is preserved, while its knowledge is extended indefinitely.

10.2 Writing ReAct-style prompts

If your framework does not provide a ReAct harness, you can still approximate the pattern with a prompt. A useful template:

> You have access to the following tools: - search(query): returns top web results - calculate(expression): returns the numerical result - read(url): returns the text of a web page For each step, output either: THOUGHT: <what you're thinking> ACTION: <tool>(<args>) or FINAL_ANSWER: <the answer> Begin. Question: [the question]

When the model outputs an ACTION, your harness parses it, runs the tool, and appends an OBSERVATION line with the result. Then the model produces the next THOUGHT. The whole transcript grows as the problem is solved. When the model outputs FINAL_ANSWER, you stop.

10.3 Why agents are hard

ReAct is conceptually simple and operationally treacherous. The loop can run forever if the model does not know when to stop. It can invent tools you did not give it, call them, and then be confused when the tool call "returns" nothing. It can interpret tool outputs wrong, building an elaborate analysis on top of a misread number. It can lose track of what it has already tried and repeat itself. Production agent systems are mostly the machinery that prevents these failure modes, not the prompt that invokes them.

The practical guidance for anyone starting with agents: begin with the fewest possible tools. A powerful agent with three tools is often

more reliable than a weak agent with twenty. Every tool is another surface for the model to misuse. Add tools only when the current set clearly fails on tasks you care about. And always log the full transcript of thoughts, actions, and observations, because the transcript is the only thing you can debug when an agent goes wrong.

Chapter 11 — Constitutional Prompting and Self-Critique

A model, asked to produce its best answer, usually produces an answer that is good. Asked to produce its best answer and then critique that answer and improve it, it often produces an answer that is better. This is one of the more counterintuitive findings of modern prompt engineering: a single pass is not the model's best work. The model's best work is its first pass plus one round of self-editing. The technique has several names. Constitutional prompting is the framing popularised by Anthropic, where the critique is done against a written set of principles. Self-critique is the more general term.

11.1 Why self-critique lifts quality

Recall from Chapter 2 that a model cannot take back a token once it has been generated. An error made early in the response commits the rest of the response to working around that error. The model is a cautious improviser, not a considered writer. But if you take the full output and feed it back in with a critique instruction, the model enters a new generation pass with the full first-pass text visible. It can now spot its own weaknesses the way a reader would, because from its position the text looks like someone else's work.

This is not imagination; the effect is robust. Self-critique catches factual errors the model made but had no chance to double-check the first time. It catches tonal drift where the response started punchy and sagged into hedging. It catches structural imbalances — a three-point argument where the third point got one sentence while the second got four paragraphs. The revised output is, on average, clearly better than the first pass.

11.2 The two-pass prompt

The simplest self-critique prompt splits the work into two calls. First call: the original task. Second call: the full first-pass output,

with an instruction like "Review the following response. List the three most significant weaknesses — factual, structural, or stylistic. Then rewrite the response to fix them. Return only the revised response."

This two-call pattern roughly doubles the cost of the prompt but lifts quality enough to be worth it on high-stakes work. For drafting important emails, analysing documents, writing code that has to run, or producing anything you will share with other people, I recommend defaulting to it. The model is a better editor than it is a writer; the self-critique pass harnesses that asymmetry.

11.3 Constitutional prompting

Constitutional prompting is a more structured variant. You write, in advance, a short list of principles that the output should conform to. "Facts must be supported by the provided context or marked as uncertain. Tone should be direct without being curt. No hedge words unless accuracy requires them. Length should not exceed 300 words." Then the critique pass explicitly checks the first draft against each principle and rewrites to conform.

The power of the constitution is twofold. First, it makes the criteria for "good" explicit and consistent across runs, which is essential for any system used at scale. Second, it surfaces tradeoffs. If the first pass is long and the constitution limits length, the revision must cut something. What the model chooses to cut tells you whether your constitution captures your real priorities or whether you need to refine it.

11.4 Debate and critique chains

For the highest-stakes work, one round of critique is sometimes not enough. You can chain further passes. Have the model critique the critique, or have a separate prompt adopt a devil's-advocate role to find weaknesses a friendly critique would miss. At a certain point diminishing returns kick in — the third pass usually adds less than the first — but for important work where you are willing

to spend multiple model calls, the chain produces noticeably more polished output than any single pass could.

Self-critique is not self-confidence

Asking a model to critique itself is NOT the same as asking it whether it is confident in its answer. The latter produces vague reassurances and is almost useless. The former produces specific, actionable revisions. Always ask 'what is weak here and how should I change it?' — never 'are you sure?'.

Chapter 12 — Retrieval-Augmented Generation (RAG)

A model's knowledge is frozen at the moment its training ended. Ours is not. If your use case involves current events, proprietary information, or any corpus too specialised or too recent for the model to have memorised, you will need to bring that information to the model rather than relying on what it already knows. The pattern for doing this is called retrieval-augmented generation, or RAG, and it is the backbone of almost every production AI system built on top of company data.

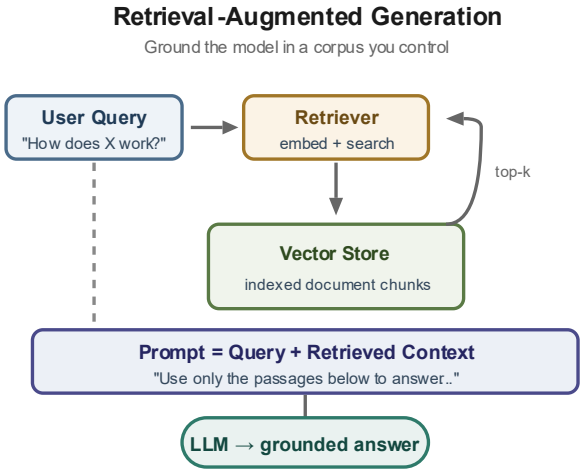


Figure 12.1 — RAG at a glance. The model gets fresh, grounded context it could never memorise, delivered just in time.

12.1 The idea

The core idea: when a user asks a question, do not send the question directly to the model. First, search some body of documents for the passages most likely to contain the answer. Take the top few passages — three to ten is typical — and paste them into the prompt along with the question. Ask the model to answer using only the provided passages. The model's answer is now grounded in your documents, not just its training.

The "search" step is usually done with vector search: every document in your corpus is converted into an embedding (a vector of a thousand or so numbers), and every question is converted into an embedding the same way. The passages whose embeddings are closest to the question's embedding are the ones retrieved. This works because related concepts cluster together in embedding space, which we met in Chapter 2.

12.2 Chunking: the most-overlooked step

The single factor that most affects RAG quality, in my experience, is how you cut your documents into chunks before embedding them. Chunks that are too large dilute the embedding — the chunk's vector represents a smeared average of many topics, and it matches questions poorly. Chunks that are too small fragment the information — answers that require two paragraphs to express get cut across two chunks, neither of which alone can satisfy the query.

The right chunk size depends on the documents. For technical documentation with short sections, chunk by section. For long-form prose, chunk by paragraph or by sliding windows of a few sentences with some overlap. For tables and structured data, often each row or logical group is its own chunk. There is no universal recipe; anyone who tells you there is selling you something.

A sensible default for text documents: chunks of 300 to 600 tokens, with a 50 to 100 token overlap between consecutive chunks. Monitor which chunks get retrieved for real queries and adjust based on where the retrieval fails. If you see repeated cases where the retrieved chunk is adjacent to the one that contained the answer, your chunks are too small. If you see retrieved chunks that are topically broad, they are too large.

12.3 The RAG prompt template

Once retrieval has happened, the prompt you build looks like this:

> You are a helpful assistant. Answer the user's question using ONLY the information in the context below. If the context does not contain enough information to answer, say so rather than guessing. CONTEXT: --- [Chunk 1] --- [Chunk 2] --- [Chunk 3] --- QUESTION: [the user's question] Answer:

The explicit instruction to use ONLY the context is essential. Without it, the model will often fold in training-data knowledge that contradicts your context, producing answers that look authoritative but are wrong for your domain. With it, the model sticks to your documents and tells you when it cannot.

12.4 When RAG fails

RAG is not magic. Common failure modes: the question is phrased in terms that do not overlap with the documents' phrasing, so the right chunks are not retrieved; the documents contradict each other and the model picks one at random; the answer requires synthesising information from many chunks and only three fit in the context; the question is a multi-hop one where the first answer is needed to formulate the second query.

Remedies for each exist. Query rewriting — using the model to rephrase the user's question into better search queries — helps with the first. Re-ranking the retrieved chunks with a second, more expensive similarity model helps with the third. Agentic RAG, where the model can issue multiple retrieval queries in sequence, helps with the fourth. The specific implementations are beyond this chapter, but the pattern is always the same: when RAG misbehaves, examine what was retrieved. The fix is usually at the retrieval step, not in the prompt.

RAG is a prompting pattern first

A great many 'RAG is broken' complaints turn out to be prompting mistakes — missing the 'use only this context' instruction, or missing chunk delimiters, or letting the model's own knowledge sneak in through a poorly-worded system prompt. Before redesigning your retrieval stack, re-read your final prompt and verify it says exactly what you want it to say.

PART FOUR

The Adaptive Secrets

• • •

Chapter 13 — Meta-Prompting: Making AI Design Its Own Prompts

The techniques in the first three parts of this book are things you do to the model. The techniques in this part are things you do with the model — patterns of collaboration in which the model becomes an active partner in the prompting itself. The first and most underused of these is meta-prompting: asking the model to help you write a better prompt.

13.1 The model is an expert in prompting

A modern language model has read more about prompt engineering than almost any human alive. Papers, blog posts, tutorials, best-practice guides, every forum discussion of what works and what doesn't — all of it is in the training data. If you ask it for advice on how to prompt it, the advice it gives is, on average, excellent. This is a strange but useful fact, and it has practical consequences.

When you have a task you are struggling to prompt, try this: paste the task description and your current prompt into a fresh conversation, and say something like "Here is the task I am trying to get done, and here is my current prompt. What is weak about this prompt? Rewrite it to be substantially better, keeping the same overall goal. Explain what you changed and why." The response will frequently be more insightful than the prompt you wrote yourself. You will see your blind spots — the layers you forgot, the constraints you left implicit, the examples you should have provided.

13.2 The prompt-writer prompt

A useful variant is to keep a standing meta-prompt — a prompt whose only job is to design other prompts. Something like:

> You are an expert prompt engineer. When I give you a task, you will: 1. Ask me up to three clarifying questions to understand the goal. 2. Once answered, propose a prompt template with all six layers (Role, Context, Instruction, Constraints, Examples, Output Format). 3. For each layer,

> explain in one sentence why you chose what you did. 4. Offer two variations of the template — one shorter, one longer — and recommend which to try first.

Using this as a first stop before real prompting work saves time in two ways. You get a better starting prompt than you would have written, and you are forced, by the clarifying questions, to articulate the task more precisely than you would have. Half the quality gain comes from the thinking the meta-prompt forces on you, not from the prompt it produces.

13.3 Prompt refinement chains

Meta-prompting can also be used iteratively. You run your prompt, see what the output looks like, and then hand the prompt and a sample of its output back to the model with an instruction: "Here is the prompt I ran and here is the output it produced. The output has the following problems: [problems]. Rewrite the prompt to fix these, without losing the strengths it currently has." The model's revision often fixes the issues in ways you would not have thought of, because it has seen, in training, a great many examples of how these specific problems get solved.

This loop — run, observe, critique, revise the prompt, repeat — is the heart of expert prompt engineering. It is less like writing code and more like refining a recipe: each iteration is cheap, the feedback is immediate, and the improvement is cumulative. The prompters who do this deliberately, with even a rough discipline about it, pull ahead quickly of the ones who do not.

13.4 The danger of over-elaboration

One caveat. Meta-prompts tend to lengthen over time. Each iteration adds a constraint here, a clarifying example there, a role refinement above. After a dozen cycles, your once-tidy prompt has become a baroque instrument that few humans can read and that the model itself starts to struggle with — the lost-in-the-middle problem again. When this happens, do not keep adding. Prune. Ask the meta-prompt to produce a minimal version that achieves

the same behaviour. Usually the minimal version can be less than half the length and perform essentially as well, because much of what accumulated was preventing failure modes that no longer occur.

Chapter 14 — The Context Window Economy

Every prompt you write is a choice about how to spend a finite budget. The context window of a modern model — anywhere from 8,000 to 2,000,000 tokens depending on the model — sounds vast, but the space fills faster than you expect, and the model's attention degrades well before you hit the formal limit. Understanding this economy, and budgeting within it, is what separates prompters who handle real workloads from those who stay in toy-problem territory.

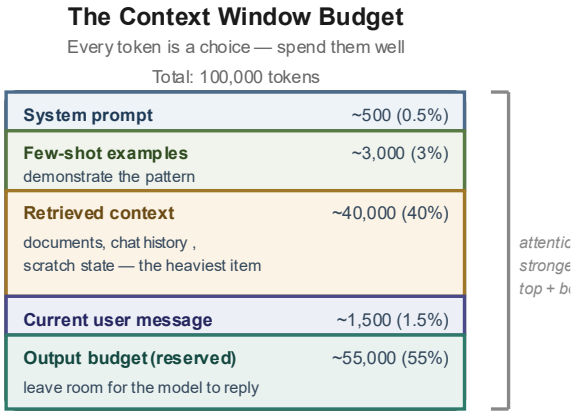


Figure 14.1 — Budgeting the context window. Each segment has a purpose; no segment should be larger than its purpose demands.

14.1 Where the tokens go

For any non-trivial prompt, the context divides into roughly six buckets. The system prompt (persona, instructions, constraints) is usually five to ten per cent. The conversation history, in a chat interface, can grow to dominate — often a quarter of the budget after a few turns. Retrieved documents, when RAG is in use, claim another fifteen to thirty per cent. The current user message is a few per cent. The model's output, which also counts against the

budget, is another ten to twenty per cent. The rest is buffer, and unless you are being deliberate about it, it evaporates fast.

Keep a rough mental accounting as you build a prompt. When you find yourself adding a fifth paragraph of context, ask whether those tokens are earning their keep. When you realise a conversation has run to thirty turns and the current question only needs the last three, summarise the rest. The goal is not to minimise context; it is to spend it on the parts of the task that actually benefit from it.

14.2 The lost-in-the-middle effect

Attention is not uniform across a long context. Study after study has shown that when important information is placed in the middle of a very long prompt, models often miss it, while the same information at the start or the end is reliably attended to. This is sometimes called the U-shape of attention, and it is one of the most practical facts in prompt engineering.

Exploit it. Put the instruction at the top and repeat it at the bottom. Put the most important source documents at the start of the retrieved-chunks block. If you have a critical constraint the model tends to drift from, mention it in the system prompt and restate it in the user message. The model is not being lazy; it is doing its best with attention that is structurally biased toward the edges. Your job is to arrange the prompt so the important things live where attention is strong.

14.3 Summarisation as a context-saving strategy

For long conversations or long documents, summarisation is often the right move. Instead of pasting the entire previous conversation into every turn, summarise the key decisions, open questions, and constraints, and paste the summary. Instead of embedding an entire document, summarise its main claims and cite specific passages on demand. Summaries cost tokens to produce, but they

cost far fewer tokens than the originals to include, and they concentrate the information in a form attention handles well.

A particularly effective pattern for long chats: after every ten turns, have a separate model call summarise the conversation into a compact paragraph that can replace the bulk of the history going forward. This maintains continuity without accumulating the bloat of a long transcript. Some agent frameworks do this automatically; in your own work, you can do it by hand.

14.4 When bigger context hurts

A counterintuitive finding: for many tasks, giving the model more context reduces performance. A prompt with two carefully-chosen examples often beats a prompt with ten loosely-chosen examples. A focused instruction beats a sprawling one. The reason is the same attention mechanism that gives the model its power: the more tokens it must weigh, the thinner each token's share of the weight, and the more likely the model is to miss the one token that actually matters.

The practical discipline, then, is to include only what the task needs. Before adding any chunk of context, ask what specifically it enables the model to do that it could not do without it. If you cannot answer that question, leave the chunk out. Prompt length is a cost centre, not a quality metric. The best prompts are often shorter than their authors first believed they needed to be.

Chapter 15 — The Iteration Loop

This chapter will not teach you a new technique. It will try to convince you, if you are not already convinced, that the most important thing you can change about your prompting is your rhythm of practice. Everything else — the layers, the examples, the meta-prompts, the constitutions — is a tool. The iteration loop is the habit through which the tools get used.

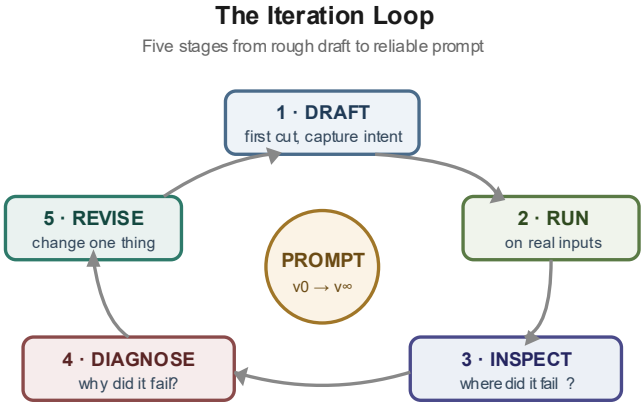


Figure 15.1 — The iteration loop. Draft, test, diagnose, refine, measure. One cycle takes minutes. Ten cycles make a professional.

15.1 The five stages

Every deliberate round of prompt improvement passes through five stages. Draft — write a first version quickly, without trying to be perfect. Test — run it against three to five real inputs that represent the variety of cases you care about. Diagnose — look at where it failed and name the failure in concrete terms ("the tone went corporate", "the JSON had extra keys", "the summary missed the deadline clause"). Refine — change one thing to address that specific failure. Measure — run the revised prompt against the same inputs and confirm the failure is fixed without creating new ones.

The whole loop, for a typical prompt, takes five to ten minutes. The discipline is in doing it at all. Most people, having written a draft that looks reasonable, skip straight to production use. The first few real inputs expose the failures, but by then the context of the prompt has faded from mind, and fixes become patchy reactions rather than considered revisions. You end up with a prompt that nobody fully understands.

15.2 Change one thing at a time

When a prompt is not working, the instinct is to rewrite half of it. Resist. Changing many things at once means you cannot tell which change mattered. If the new version is better, you do not know which of your edits was the improvement and which was the regression you compensated for. If it is worse, you have no clean starting point to back up to. The iteration loop works because each cycle is a small, controlled experiment.

Keep a short log. Date, the change you made, the result. A few lines per iteration. After a dozen cycles on an important prompt, the log becomes a record of what actually works for this kind of task — worth more, over time, than the prompt itself, because it transfers to every similar task you tackle next.

15.3 The test set

For any prompt you will reuse, assemble a small test set — five to ten representative inputs, ideally including the cases you have seen fail before. Run every prompt revision against the whole set, not just the input of the moment. This prevents the classic failure mode where fixing one kind of error silently breaks another.

The test set need not be sophisticated. For a prompt that summarises customer emails, five or six real emails spanning the types you see (a complaint, a compliment, a billing question, a feature request, a confused reply) are enough. The discipline is in running all of them every time. Most prompt engineers do not. The ones who do produce prompts that are noticeably more reliable.

15.4 Knowing when to stop

The loop could, in principle, run forever. Every prompt can be made a little better. The skill is in recognising diminishing returns and stopping. A useful heuristic: once three consecutive iterations fail to produce a meaningful improvement, declare the prompt good enough. Ship it. The time you would have spent on the tenth iteration is almost always better spent on a different problem.

The other thing to notice is when a prompt is not converging. If every fix creates a new failure, the prompt is probably being asked to do too many things at once. The correct move is not more iteration on a single prompt but decomposition — splitting the task into two or three prompts that each do one thing cleanly and chaining them together. We will see this pattern throughout Part V.

Chapter 16 — Controlling Tone, Voice, and Style

The content of a model's response is usually what people focus on first. The texture — the tone, the voice, the rhythm, the register — is what determines whether the response can actually be used. A factually correct email written in bureaucratic prose is unsendable. A witty tweet written in academic sentences falls flat. Controlling texture is as important as controlling content, and it is done with a different set of techniques.

16.1 Tone

Tone — the emotional quality of writing — is one of the easiest things to control and one of the most often neglected. State it explicitly and the model will obey. "Warm but professional." "Direct without being curt." "Gently funny." "Cool and technical." "Reassuring, like talking to a worried friend." Each of these produces a different output from the same prompt.

When you want fine control, describe the tone in two or three dimensions at once. "Direct and dry — no filler words, no hedging, no enthusiasm." The model composes against the combination. For most work this is enough. For higher-stakes work, combine the description with one or two example sentences in the target tone; the example does what descriptions cannot and pins the tone precisely.

16.2 Voice

Voice is the harder cousin of tone. Voice is the particular way a specific writer sounds — their rhythms, their tics, their images, their characteristic moves. Replicating an arbitrary writer's voice from a description alone is difficult. Replicating it from examples is much easier. Give the model three or four paragraphs of the writer's prose, tell it to study them, and ask it to write new text in the same voice. The results are often uncanny.

For ongoing work with a consistent voice — a blog, a brand's email, a publication's house style — create a voice guide. This is a document of a thousand words or so that contains: a description of the voice in two paragraphs, a list of five rules the voice always follows and five things it never does, and three or four short illustrative passages. Include the voice guide in the system prompt for every piece of writing. Output consistency becomes, with this one move, dramatically better.

16.3 Style

Style is the set of structural choices: sentence length, paragraph length, use of lists, punctuation habits, vocabulary range. Style can be specified directly. "Use short sentences. Vary paragraph length. Avoid semicolons. Prefer common words to fancy ones. Use no more than one adverb per paragraph." Each of these is a concrete rule the model can follow.

A style discipline that produces surprisingly good results: ask the model to write at a specific reading age. "Write at a Grade 8 reading level" produces prose that a thirteen-year-old can read — short sentences, common vocabulary, concrete examples. "Write at a university level" produces dense paragraphs with abstract claims. Between these poles is most of the useful range, and picking a point on it tends to solve a lot of "this reads wrong" problems at once.

16.4 When tone and content fight

Sometimes the tone you want conflicts with the content you need to deliver. Bad news in a warm voice risks sounding fake. Technical depth in a casual voice risks being imprecise. When these tensions show up, name them in the prompt. "Deliver the bad news directly, without softening the facts. Then, in a separate short paragraph, be warm about what comes next." Explicitly splitting the texture into segments lets each serve its purpose without contaminating the other.

The Three-Sample Test

Before using any new tone or voice prompt in production, run it three times against the same input. If the three outputs sound meaningfully different from each other in ways that cross your acceptable range, the prompt is under-specifying. Add more anchoring examples, or a more precise tone description, until the three outputs sound like siblings rather than strangers.

Chapter 17 — Taming Hallucinations

A language model, asked a question it does not know the answer to, will often produce an answer anyway — fluent, confident, and wrong. This phenomenon, called hallucination, is the single most frequently cited weakness of modern AI, and for good reason: it can be catastrophically misleading, especially when the reader is not in a position to check. But hallucinations are not random noise. They have causes, patterns, and — more hopefully — countermeasures. This chapter covers the ones that work.

17.1 Why models hallucinate

Recall the core mechanism from Chapter 2: the model predicts the most plausible next token given the context. "Plausible" is statistical, not epistemic. If the training data contains many sentences that sound like the one the model is producing, it produces the sentence — regardless of whether the sentence happens to be true. A model asked to cite an academic paper it has not seen will often cite one that sounds like what such a paper might be called, with an author name that sounds like the kind of author who would write it, in a journal that sounds plausible. Every element is a statistically reasonable guess. The combination is a lie.

This means hallucinations are concentrated in specific kinds of output. Citations, URLs, specific numbers, direct quotes, technical details the model encountered rarely during training — these are the high-risk zones. Broad conceptual explanations of things the model has seen discussed a million times are much lower risk. Knowing where the risk is concentrated is the first step toward managing it.

17.2 Grounding

The most effective defence is to ground the model in source material. If you need the model to write about a specific document, paste the document into the prompt. If you need it to cite sources,

use RAG (Chapter 12) so the citations are from a real corpus you control. If you need factual claims, require the model to quote the passage that supports each claim or mark it as uncertain. Grounded output is checkable; ungrounded output is a bet.

A particularly effective ground-truth prompt pattern: "Answer the question using only the information in the provided context. For every claim, quote the specific sentence from the context that supports it. If the context does not contain enough to answer, say so explicitly rather than filling in." This forces the model into a checkable format: every claim has a receipt, or the claim is not made.

17.3 Calibration prompts

Even without external sources, you can reduce hallucinations by instructing the model to be honest about its uncertainty. "For each statement, add a confidence level: HIGH, MEDIUM, or LOW. If LOW, explain what would be needed to raise confidence." This is imperfect — the model's self-assessment is noisy — but it is substantially better than the alternative, which is confident tone applied uniformly to everything.

Another useful instruction: "If you are not sure, say 'I don't know' and stop. Never invent citations, quotes, statistics, names, or URLs. It is better to provide less information correctly than more information incorrectly." Simply stating this shifts the model's behaviour. The "never invent" phrase, in particular, is worth keeping in your standard system prompt for any work where invention would be harmful.

17.4 Verification passes

For the highest-stakes work, treat the model's output as a draft to be verified, not an answer to be trusted. A simple verification pass: take the output, give it to a fresh model call, and ask "Check each factual claim in the following text. For each, state whether you believe it to be true, false, or uncertain, and explain why. Do not

defer to the original text; check each claim independently." The verification pass catches a surprising fraction of hallucinations because the model approaches the claims cold, without the narrative momentum that produced them.

For truly critical work — medical information, legal drafts, anything where a wrong fact could cause harm — automated verification is not enough. A human in the loop is necessary. But the verification pass still helps: it surfaces the claims most worth double-checking, so the human's time is spent where it matters most.

17.5 The last-resort constraint

When nothing else is working and the model keeps inventing things, there is a blunt instrument that almost always helps: forbid the relevant category of output. "Do not include any specific numbers, dates, names, citations, or URLs. If the task requires one of these, say 'I cannot provide that accurately' and stop." This produces less useful output in general but completely eliminates a class of failure mode. For some use cases that tradeoff is the right one, and it is better to ship a blunt-but-safe tool than a polished one that occasionally lies.

Hallucinations are not moral failures

The model is not trying to deceive you. It is, at every token, doing its best job of pattern completion with the information available. Hallucinations are what happens when the pattern-completion is untethered from truth. The defence is always architectural: structure the prompt so that truth-tethering is either unnecessary or enforced. Asking the model to 'try harder' never works. Changing what the model is being asked to produce always can.

PART FIVE

Applied Prompting

• • •

Chapter 18 — Prompting for Writers and Authors

> *"Writing is thinking. A good prompt is a good question posed to thinking itself."*
>
> — — paraphrased from many working authors

Writers were among the earliest serious users of large language models, and writers have developed the most sophisticated folk-knowledge about how to work with them. Some of that folk-knowledge is correct. Some of it is cargo-cult ritual. This chapter sorts one from the other. If you write for a living, or write seriously as a practice, the next few pages should change how you use models.

18.1 The danger of generic voice

The default voice of a large language model is a kind of competent mid-Atlantic professional-article tone. It is grammatically correct, structurally neat, and utterly characterless. If you ask a model to "write a short story" with no further guidance, it will produce something that reads like the median of every short story on the internet: tidy, sentimental, full of predictable turns. This is not the model's fault. You did not tell it what kind of writer you are.

The single highest-leverage move for a writer using a model is to feed it substantial samples of your own prose before asking it to produce anything in your voice. Not a sentence or two — a thousand words minimum, ideally several thousand. Paste a full chapter, a full essay, a representative blog post. Then ask the model to produce new work in the same voice. The difference is not subtle. The model will begin echoing your sentence rhythms, your vocabulary, your structural preferences. It will still not be you, but it will be much closer than the default.

18.2 The model as sparring partner

The most valuable use of models for serious writers is not generation but critique. The blank page problem is real, but the

blank page is rarely the biggest problem in a writing project. The bigger problem is the revision pass: what is not working, what is dragging, where the structure has broken down, where the voice has gone slack. Models are unusually good at this work, and they are good at it for reasons that are almost counterintuitive — because they do not have an ego invested in the text, they will point at problems that a polite human reader will not.

> You are a developmental editor reviewing a draft chapter from a work-in-progress novel. Your job is to identify structural problems, pacing issues, unclear character motivation, and any passages that feel forced or overwritten. Be specific. Point to sentences or paragraphs by their first few words. Do not praise — I know what is working. Tell me only what is not working, and for each problem, suggest the smallest possible fix.\n\n[DRAFT CHAPTER HERE]

Notice the structure of that prompt. It assigns a role (developmental editor), it defines the scope narrowly (structural problems, pacing, motivation, overwriting), and — crucially — it forbids the behaviour you do not want ("do not praise"). Models default to a sandwich structure where every criticism is wrapped in compliments. For serious editorial work this is worse than useless; the useful information is buried in filler. Telling the model explicitly to skip the filler is essential.

18.3 Structural work before prose work

A recurring mistake among writers new to models is to jump straight to prose generation — "write me a chapter where X happens." The resulting prose is usually unusable because the structure underneath it is generic. A better sequence is to do the structural work first, iteratively, with the model as a sounding board, and only descend to the prose level once the structure is sound.

For a chapter, this might mean: first, write a one-sentence description of what the chapter does. Second, break that into a five-beat outline of scenes or sections. Third, for each beat, write two or three sentences about the emotional arc and the specific moment that carries the weight. Only at the fourth stage do you

generate or write actual prose, and by that point the structural scaffolding is strong enough that the prose can be good. The model is useful at every stage, but it is most useful when you ask it structural questions at the structural stage and prose questions at the prose stage.

18.4 The voice-preservation workflow

A specific workflow worth adopting for writers who use models to generate draft material but want the final product to sound like them: generate a messy draft with the model, then rewrite the draft by hand. Do not edit. Rewrite. Read each paragraph, close the file, and write the paragraph again from scratch in your own words. This takes almost as long as writing from scratch, but it is easier because the structural decisions are already made, and the final output is genuinely in your voice.

If you want to preserve voice while keeping more of the model's draft, a useful middle-ground prompt is: "Rewrite the following passage to sound more like the voice sample below. Keep the same information and structure, but change word choice, sentence rhythm, and idiom to match the sample. Do not add ideas; do not remove ideas." This is an imperfect tool — the model will drift toward its default voice over long passages — but for paragraph-scale work it is effective.

18.5 What models are still bad at

Models are bad at genuine surprise. They are statistical engines; they produce the likely next thing. Great fiction and great essay-writing often turn on the unlikely next thing — the image that nobody else would have reached for, the argument that inverts the reader's expectation, the joke that lands because it comes from an angle nobody was watching. Do not expect a model to deliver these. Expect yourself to deliver them, and use the model to clear the ground so that your attention can be spent where your attention matters.

Models are also bad at sustained emotional truth. They can produce emotionally effective sentences, paragraphs, sometimes scenes. They are unreliable at sustaining emotional truth over the length of a chapter, and they struggle over the length of a book. The structural scaffolding of a long emotional arc has to be held by the writer. The model can help at the sentence level and the scene level, but the author holds the shape.

The hierarchy of writing with a model

Use the model for critique more than for generation. Use it for structural work before prose work. Feed it substantial samples of your voice before asking for anything in your voice. Rewrite by hand whatever you want to keep. The model is a powerful collaborator, but it is a collaborator without a sense of stake, without a memory of what you are trying to build, and without the ability to be genuinely surprising. Keep those functions for yourself.

Chapter 19 — Prompting for Code and Engineering

> *"The best programmers are the ones who can ask the clearest questions. The same is now true of the best people using programming assistants."*
>
> —

Code is a domain where prompting has been transformed more dramatically than almost any other. Five years ago, a programmer using an AI assistant meant autocompletion of variable names. Today, entire features are designed and implemented through dialogue, and the quality of that dialogue is the primary bottleneck. This chapter is a field manual for the working engineer — not a tutorial on a specific tool, but a set of patterns that apply regardless of which assistant is in front of you.

19.1 Give the model the context the team has

The single most common failure mode when engineers prompt models is underspecification of context. You know what codebase you are in. You know what patterns the team follows. You know which library is being used and which version. You know what the error is actually about because you saw it happen. The model knows none of this. When it produces code that looks right but breaks, the reason is almost always that it was writing code for a different, imagined codebase — one consistent with the fragmentary context you gave it.

The discipline is to over-share context ruthlessly. Paste the surrounding function. Paste the imports. Paste the type definitions. Paste the error message in full, including the stack trace. Paste the test that is failing. If the project has a conventions document, paste that too. A prompt that is 90% context and 10% question produces dramatically better code than a prompt that is 10% context and 90% question, and the time spent assembling the

context is less than the time spent debugging code written without it.

19.2 State the interface first

When asking a model to implement a function, specify the interface precisely before asking for an implementation. What does it take in? What does it return? What are the edge cases? What should it do on failure? A common pattern that works well:

> I need a function with this interface:\n\nfunction computeMonthlySummary(\n transactions: Transaction[],\n month: YearMonth\n): MonthlySummary\n\nWhere:\n- Transaction = { id: string, amount: number, date: Date, category: string }\n- YearMonth = { year: number, month: number } // month is 1-12\n- MonthlySummary = { totalIncome: number, totalExpenses: number, byCategory: Record<string, number>, transactionCount: number }\n\nBehaviour:\n- Income is transactions with positive amount; expenses are transactions with negative amount.\n- Expenses in byCategory should be stored as positive numbers (absolute values).\n- Transactions outside the given month are ignored.\n- If there are zero transactions in the month, return the summary with all zeros and an empty byCategory.\n\nWrite the function in TypeScript. Include JSDoc comments. Do not import anything beyond what is needed.

The model produces good code from this kind of prompt because there is nothing for it to guess about. Every significant decision has been made by you. The model is doing the mechanical work of translating a specification into syntax, which is exactly the work it is good at. The design work — which is where mistakes are most expensive — stays with you.

19.3 Test-first prompting

A workflow that produces remarkably reliable code: ask the model to write the tests first, review the tests carefully, then ask it to write the implementation that passes those tests. Reviewing tests is easier than reviewing implementations — tests describe behaviour directly, and their correctness is easier to evaluate. Once you are confident in the tests, the implementation is heavily constrained. Any implementation that passes the tests is probably correct, and if the tests miss a case, that is a review failure that would have happened anyway.

The test-first workflow has a further advantage: it produces artifacts you can keep. The tests remain in the codebase as documentation of the intended behaviour. Future changes can be validated against them. The cost of the extra prompting round is more than paid back by the test coverage you now have for free.

19.4 Debugging dialogues

When using a model to debug, the temptation is to paste the error and say "fix this." The temptation should be resisted. A much more effective pattern is to describe what you expected to happen, what actually happened, and what you have already tried. Then ask the model to list the five most likely causes, ranked by probability, with the evidence for each. Only after you have discussed the hypotheses should you attempt a fix.

This slower pattern catches a failure mode that the "fix this" pattern does not: the model confidently implements a fix for the wrong cause. The fix often makes the error go away — by covering it up with different code — while the actual bug remains, now harder to find. Ranking hypotheses forces the model to actually reason about the problem, and it forces you to engage with that reasoning rather than accepting a patch.

19.5 Agentic coding and the new contract

The most powerful modern coding assistants operate agentically: they can read files, run tests, execute code, observe the results, and iterate. This changes the contract of prompting. You are no longer asking the model to produce code that you will then run; you are asking the model to produce code that runs, and then to use the results of running it to improve itself. The prompt you write is less like a specification and more like a mission briefing.

Effective mission briefings for agentic assistants follow a recognisable pattern. State the goal in terms of observable outcomes ("all tests in src/payments pass", "the homepage loads without console errors"), not in terms of code changes. State the

constraints ("do not modify files in src/legacy", "do not add new dependencies"). State the verification criterion ("run npm test after every change and do not declare done until every test passes"). With this structure, the agent has a clear termination condition and a clear set of rails. Without it, the agent will wander.

19.6 What engineers should never delegate

Architectural decisions. Security-critical choices. Anything involving authentication, authorisation, cryptography, or data that should not leak. Anything where the consequence of being wrong is catastrophic and hard to reverse. Models are acceptable implementers of well-specified crypto; they are terrible designers of new crypto. They are fine at implementing a password reset flow you have designed; they are dangerous at designing one. The rule is: the model implements; the engineer owns the design. Owning the design means being able to explain why every significant choice was made. If you cannot explain it, you do not own it, and you should not ship it.

The engineer's prompt stack

Lead with context. Specify interfaces before asking for implementations. Write tests first when reliability matters. Rank hypotheses before fixing bugs. For agents, brief missions rather than specify code. And never, ever let the model own a decision whose consequences you cannot explain.

Chapter 20 — Prompting for Research and Analysis

> *"A good analyst does not answer questions. A good analyst finds the question that, once answered, makes the original question obvious."*
>
> —

Research and analysis work with models is a minefield of plausible-sounding nonsense. The model is fluent; the model writes like an analyst; the model produces output that looks exactly like the output a careful research team would produce. The output is often wrong. This chapter is about how to use models for research work without being deceived by your own tools.

20.1 The fluency trap

The central problem of model-assisted analysis is that fluency is a poor proxy for correctness, but a powerful trigger for trust. When a model produces a confident-sounding three-paragraph answer with specific numbers and an apparent source, the human reader's default is to treat it as reliable. Often it is not. The specific number may be invented, the source may not exist, the logical chain may have a broken link buried in the middle. The reader's fluency-heuristic is mis-calibrated because until recently, only competent analysts could produce that kind of prose; now, anything can.

The defensive stance is to treat every model-produced factual claim as a hypothesis rather than a fact. The prompt should make this explicit. "For each claim, provide the source. If you do not have a verifiable source, mark the claim [UNVERIFIED] and explain what evidence would be needed to verify it." With this instruction, the output becomes checkable. Without it, you are trusting a system whose core operating mechanism is plausibility, not truth.

20.2 Decompose before you delegate

Large research questions decompose badly when handed to a model whole. "Analyse the impact of remote work on commercial real estate" is a real research project with many sub-questions, and the model's answer will average over them unhelpfully. A better workflow is to decompose the question first — either yourself or with the model's help — into a tree of sub-questions, then tackle each sub-question in its own prompt with its own context.

The decomposition prompt: "I am researching [topic]. Before attempting to answer, list the sub-questions I would need to answer first to produce a complete analysis. Organise them as a dependency tree — which sub-questions must be answered before others can be attempted. For each sub-question, note what kind of evidence would answer it." The output of this prompt is not an answer; it is a research plan. The plan is what you then execute, one sub-question at a time, with sources and sanity-checks along the way.

20.3 Steelman, then stress-test

A pattern that produces unusually high-quality analytical work: first ask the model to build the strongest possible case for a position, then ask it to build the strongest possible case against that position, then ask it to identify where the two cases actually conflict versus where they are talking past each other. The third step is the hardest and the most valuable, and it almost never happens without being asked for explicitly.

Many debates that look like disagreements about facts are actually disagreements about which facts are relevant. The "where do they actually conflict" prompt surfaces this. Often the answer is that the two positions are mostly compatible once you clarify the implicit assumptions, and the real disagreement is a narrow empirical question that could be resolved by looking at specific evidence. This kind of clarification is what good analysis produces, and

prompting for it directly is far more efficient than hoping it will emerge.

20.4 Numerical caution

Models are unreliable with numbers. They are unreliable with them in specific, predictable ways: they invent specific-sounding figures, they confuse units, they do arithmetic wrong, they confabulate growth rates and ratios. The defensive posture is to require every number to come from a source the model can cite or from a calculation the model shows explicitly. "Whenever you cite a number, either provide the source (which I will verify) or show the arithmetic step by step (which I will check)." This is slower than asking for the number directly, but the number you get is one you can actually use.

For serious quantitative work, the further discipline is to keep the model out of the arithmetic entirely. Use the model to describe the calculation; do the calculation in a spreadsheet or a script you can inspect. The model is an excellent translator from prose to formula and back; it is a poor calculator. Keep each tool on the work it is good at.

20.5 Synthesis across sources

For literature review and cross-source synthesis, the most useful pattern is to feed the model one source at a time, asking for a structured extract, before asking for synthesis. "For the following paper, extract: (a) the central claim, (b) the evidence offered, (c) the main limitations acknowledged by the authors, (d) the main limitations not acknowledged, (e) how the claim relates to [your research question]." Do this for each source, save the extracts, then feed all extracts to a final synthesis prompt.

This pattern works because it forces the model to engage with each source individually rather than blending them into a generic summary. It also produces artifacts — the structured extracts — that are useful independent of the final synthesis. If the synthesis

goes wrong, the extracts are still valuable. If the synthesis goes right, the extracts are the audit trail that lets a reader check the work.

Research hygiene in one sentence

Treat every model-produced fact as a hypothesis, every model-produced number as suspect, and every model-produced synthesis as an argument that needs sources — and build your prompts so that the sources, hypotheses, and arithmetic are surfaced and checkable rather than hidden inside fluent prose.

Chapter 21 — Prompting for Business and Marketing

> *"The most expensive words a business can produce are the ones nobody reads."*
>
> —

Business and marketing work is the domain where models are most widely deployed and most frequently misused. The same tool that can draft a genuinely persuasive landing page will happily produce a thousand words of indistinguishable corporate mush if you do not tell it otherwise. The skill is in the telling. This chapter is about the prompts that separate useful business writing from content-shaped noise.

21.1 The specificity principle

The default voice of a model prompted for business writing is the voice of every business-writing training corpus it ever saw: committee-approved, risk-averse, generic. This voice is the sound of nothing being said. Every sentence is true of any company doing anything roughly similar. No reader will remember a word of it, and the writer has wasted everyone's time by producing it.

The antidote is aggressive specificity. Every claim in a piece of business writing should either be specific or be cut. "We help businesses grow" — cut. "We help Shopify merchants recover 18% of abandoned carts within 30 days" — keep. The prompt that produces the second kind of sentence looks nothing like the prompt that produces the first. It has to include the specifics: the target customer, the specific outcome, the specific timeframe, the specific evidence. If those are not in your prompt, they cannot be in your output.

21.2 The 'who is this for and why should they care' frame

A prompt structure that reliably produces sharper marketing copy: before asking for any output, describe the target reader in one paragraph (who they are, what they are trying to do, what is frustrating them, what they have already tried), and describe in one paragraph what would make them stop scrolling and pay attention. Only then ask for the copy.

> Target reader:\nIndependent Shopify store owners, usually solo founders or small teams, doing $50k-$500k in annual revenue. They are trying to grow their email list without paying for ads. They have tried pop-ups but found them low-converting and annoying. They have tried content marketing but do not have time to write consistently.\n\nWhat would make them stop scrolling:\nA specific, concrete promise about a mechanism they have not tried, with evidence that it actually works, from someone who clearly understands their world (not a generic 'grow your business' pitch).\n\nTask:\nWrite a 120-word landing page hero section for [product]. Lead with a specific outcome, not a feature. Use language these founders actually use, not marketing language. Avoid: 'empower', 'unlock', 'transform', 'revolutionise', 'leverage', 'seamless', 'cutting-edge'.

The forbidden-word list at the end of that prompt is not a joke. It is the single highest-leverage line in the entire prompt. Generic business writing is built from a small recurring vocabulary of dead metaphors and filler verbs. Ban them, and the model is forced to reach for something more specific. What it reaches for is usually much better than what it would have produced by default.

21.3 The strategy-before-copy discipline

A persistent mistake in marketing work with models is to ask for copy before the strategy is settled. The model will happily produce copy for any strategy, so the copy looks good in isolation. It is often serving the wrong strategy, and no amount of polish on the copy will fix that.

A better sequence: first use the model to sharpen the positioning — what category are we in, who is our reference customer, what are the three alternatives they are currently considering, why would they pick us over each one. Second, use the model to articulate the single most important message we need this audience to internalise. Only third, once those two layers are clear, ask for copy. Copy produced on this foundation is dramatically

easier to write because almost every decision has already been made.

21.4 Segmentation through prompting

Good marketing is not one message, it is the right message to each segment. Models make it economically viable to produce many versions of the same piece of writing, each tuned to a specific audience. The prompt pattern: "Here is the core message: [message]. Produce five variations of this landing page hero, each tuned to a different customer segment. For each, first state who the segment is, what their specific concerns are, and what language they use. Then produce the hero copy."

This is work that would have been unaffordable for most businesses five years ago — producing five fully tuned variations of every piece of marketing was an agency-budget activity. Now it is a single prompt. The constraint is no longer cost; it is the attention to specify the segments well enough that the variations actually differ. Without rich segment descriptions, the five variations will collapse into paraphrases of the same generic pitch.

21.5 The internal-communication use case

Business writing is not only external. A large and usually underestimated use of models is for internal communication: memos, updates, proposals, decision documents. Here the voice demands are different — you are not trying to sell, you are trying to be understood quickly by busy colleagues. The highest-value prompt pattern: "Rewrite the following draft to front-load the conclusion, reduce the word count by 40%, and remove any sentence that a reader could skip without losing the point."

Most first drafts of internal documents bury the conclusion under context. Most readers make a decision about whether to keep reading in the first two sentences. A ruthless front-loading rewrite is almost always an improvement, and it is one of the tasks models are genuinely excellent at. The before-and-after on a typical

internal memo is striking: what was an 800-word document that nobody finished becomes a 350-word document that everyone reads. The information content is the same. The delivery is completely different.

The marketer's and operator's prompt principles

Specific beats general. Strategy before copy. Segment before sending. Front-load conclusions. Ban the filler words. And remember: the model will produce whatever quality you demand. If you accept mush, you will get mush. If you demand specifics, you will get specifics. The prompt is the standard; the output follows the standard.

Chapter 22 — Prompting for Education and Learning

> *"The best tutor is one who knows exactly what you don't know and explains only that."*
>
> —

Education is possibly the domain where well-prompted models will have the largest impact, and where badly prompted models will do the most damage. A model that teaches well can personalise to the learner in a way that no textbook and few teachers can. A model that teaches badly produces a confident-feeling learner who has memorised patterns and cannot apply them. The difference between these two outcomes is almost entirely a matter of prompt design.

22.1 The 'do not tell me the answer' pattern

The single most important learning-oriented prompt is the one that stops the model from giving you the answer. By default, if you ask a model "what is the derivative of $x^2 \sin(x)$?", it will tell you. For learning, this is the wrong response. You learn nothing from being told. The prompt that produces learning is: "Do not give me the answer. Instead, ask me a question that helps me work out what step to take first. If I give a wrong answer, do not correct me directly — ask me a question that helps me see my mistake."

This is a genuinely different mode of interaction, and it is uncomfortable the first few times because it is slower than being told. But "slower" is the signal that actual learning is happening. When the model answers directly, you get an illusion of progress; when the model makes you do the work, you get the work. For any topic you are trying to actually learn rather than merely look up, this prompt pattern is the one to internalise.

22.2 The adaptive-explanation prompt

A model can adjust the level of an explanation to a learner in a way no textbook can. The prompt that unlocks this: "Explain [concept] at my level. I will then tell you what I understood and what I did not. Adjust your next explanation to target the specific things I did not understand. If my confusion is because I am missing a prerequisite concept, stop and explain the prerequisite before returning to the main topic."

The critical clause is the last one — the acknowledgement that confusion is often a prerequisite-gap, not a failure of the current explanation. Almost every hard topic is hard because one of its prerequisites is not solid. A teacher who keeps re-explaining the current topic when the gap is upstream is a teacher who loses students. The prompt above makes the model behave like a teacher who notices the gap and fixes it.

22.3 The spaced-practice prompt

Learning science has shown for a century that spaced practice produces vastly better long-term retention than massed practice. Models can implement spaced practice in a way that is extremely lightweight: "I am learning [topic]. Generate three questions at my current level. I will answer them. Then generate three more questions that (a) test the same concepts differently and (b) introduce one new concept that builds on what I just demonstrated. Continue this pattern. Every fifth round, include a question about something I covered several rounds ago, to check retention."

This prompt converts the model into an on-demand quiz engine with built-in retention testing. For the learner, it is much more effective than re-reading notes. For the cost of a few minutes of prompting, the learner gets a quiz bank sized to their specific curriculum, adaptive to their specific weaknesses, with retention testing built in. This is pedagogical infrastructure that would have been unthinkable ten years ago.

22.4 The explanation-back prompt

The Feynman technique — if you cannot explain something simply, you do not understand it — can be made interactive with a model. The prompt: "I just learned [concept]. Let me explain it back to you as if I were teaching a smart twelve-year-old. After I finish, point out any place my explanation was vague, wrong, or missing an important piece. Do not be generous; be accurate."

The value of this prompt is that it exposes the learner's own gaps. Most learners, tested by re-reading, think they understand something; tested by explaining it aloud, discover they do not. The model's critique is the missing piece of the technique — Feynman's original version required a real listener who could tell you when you had failed, and most self-learners do not have access to that. The model does that job cheaply and patiently, which is exactly the combination the learner needs.

22.5 The trap of learning without struggle

A warning that applies to every pattern in this chapter: it is possible to use models in a way that feels like learning but is not. If you ask a model every question as it arises, you build a dependency rather than a skill. If you accept every explanation without producing it back in your own words, you accumulate words without understanding. If you skip the productive struggle of not knowing, you skip the part of learning that actually produces learning.

The discipline is to use the model as a prod rather than a pacifier. Ask it for questions more than for answers. Ask it to check your explanations more than to provide explanations. Tolerate being wrong and corrected, many times, as the actual texture of the work. A learner who can do this will outpace a learner who simply asks the model to explain everything, even if the second learner spends ten times as many hours. The mechanism of learning is struggle; the tool matters less than whether the tool is used to enable struggle or to avoid it.

Teach yourself, with the model as your sparring partner

The highest-value learning prompts withhold rather than provide. They ask you questions rather than answer yours. They check your explanations rather than offer their own. Used this way, a model is the best tutor you have ever had: infinitely patient, available at three in the morning, deeply expert, and willing to follow your specific confusions exactly as far as they go. Used the wrong way, it is a crutch that feels like wings.

PART SIX

The Future of Prompting

• • •

Chapter 23 — Agents and Autonomous Workflows

"An agent is just a prompt that keeps running."

—

The most significant shift in how models are used over the past two years is the move from single-turn completions to agentic workflows. Instead of a prompt producing an answer, a prompt produces a plan, and the model then executes the plan by taking actions — reading files, running code, searching the web, calling APIs — observing the results, and deciding what to do next. The prompt you write for an agent is doing much more work than the prompt you write for a chat completion, and writing it well is a distinct skill.

23.1 What changes when the model can act

The core shift is from answering questions to pursuing goals. A chat completion ends when the model produces text; an agent loop ends when a goal is achieved or abandoned. This means the prompt must specify not only what you want but how the agent should know when to stop, how it should handle failure, what actions it is allowed to take, and what it should report back when finished. Missing any of these produces a familiar failure: the agent wanders, tries increasingly desperate approaches, or declares success on a task that was only partially done.

The prompt template for a reliable agent has a predictable shape: the goal, stated in terms of observable outcomes; the allowed tools and any constraints on their use; the definition of done; the reporting format. A goal of "fix the failing test" is ambiguous. A goal of "make the test in src/tests/payments.test.ts pass without modifying the test file itself or adding mocks, and report back the exact changes made" is actionable. The difference in reliability between these two framings is enormous.

23.2 The planning-execution split

A pattern that produces strikingly better agent behaviour: separate planning from execution. In the planning phase, the agent is asked to produce a step-by-step plan, identify potential failure modes, and describe the evidence it will gather to verify success. The plan is then either approved (by the user or by another model acting as reviewer) before execution begins, or it is used as a commitment that constrains the execution phase.

Without this split, agents tend to plan-as-they-go, which means they drift under pressure. When a step fails, they invent new approaches that were never part of the plan, sometimes in contradiction to the constraints. With an explicit plan, when a step fails, the agent returns to the plan and either finds a different path within it or reports a blocker. The first behaviour produces bugs that are hard to trace; the second produces clear failures that can be diagnosed and retried.

23.3 Tool design is prompt design

When you give an agent tools, the description of each tool is part of the prompt. A tool called search_docs with the description "search the documentation" is a different prompt from a tool called search_docs with the description "search the internal engineering docs (not customer-facing documentation) — returns markdown snippets with file paths. Use this tool when you need to find how a specific function, module, or internal concept is implemented. Do not use for general web search." The second description shapes the agent's decision about when to use the tool. The first description leaves that decision to chance.

Good tool descriptions include: what the tool does, what inputs it takes, what outputs to expect, when to use it, when not to use it, and any gotchas. They are written for the model, not for a developer reading an API reference. Treat tool descriptions as high-leverage prompting surface, because that is what they are.

23.4 Context management over long loops

Agents run for many steps, and the context window fills up. How the agent manages its own context determines whether it can complete long tasks or collapses halfway through. Two patterns are essential. The first is summarisation: as the context approaches its limit, the agent compresses the history of what it has done so far into a much shorter summary that preserves the essential state (what the goal is, what has been accomplished, what is blocked, what to try next). The second is externalisation: state that does not need to be in the context should be written to a file or a scratchpad and re-read when needed.

Prompts for long-horizon agents should include explicit instructions on context management. "Every ten tool calls, summarise your progress so far into a single paragraph and save it to progress.md. When context starts filling up, rely on progress.md as your source of truth about what has been done." This discipline is the difference between agents that can complete a multi-hour task and agents that succeed for the first hour and then forget what they were doing.

23.5 When to use an agent and when not to

Agents are powerful and correspondingly expensive, in tokens, latency, and risk. They are appropriate when the task requires iteration against feedback — the agent tries something, observes the result, adjusts. They are appropriate when the task involves many small steps that would otherwise require many round-trips with a human. They are inappropriate when the task is short and well-specified enough that a single prompt would do it faster. They are inappropriate when the cost of a mistake is high and the agent's actions are hard to audit.

A useful heuristic: if a human expert could do the task faster than it takes to write a good agent prompt, do the task. If the task involves more than a few rounds of try-observe-adjust, and the stakes are contained (you can review the result, nothing irreversible happens), an agent is worth it. For anything in

between, a well-crafted non-agent prompt with explicit steps is often the right middle ground.

The agentic mindset

An agent is a prompt that produces a plan and executes it. The quality of agentic work is the quality of the goal, the quality of the tools, the quality of the context management, and the quality of the stop condition. Specify all four, and you have a reliable automation. Skip any of them, and you have an expensive random walk.

Chapter 24 — Multimodal Prompting

> *"The text is the ocean. Images, audio, and video are continents that are still mostly uncharted."*
>
> —

For most of the history of large language models, "prompting" meant "text". That is no longer true. Modern models accept images, produce images, handle audio, process video, and combine these modalities in ways that are only starting to be explored. This chapter is a short introduction to the prompting patterns that are emerging for multimodal work.

24.1 Image-in, text-out

The most common multimodal pattern today is giving a model an image and asking for analysis or description. The first-order lesson is that models are vastly better at structured image queries than at open-ended ones. "Describe this image" produces a generic description. "What is the specific plant visible in the bottom left of this image, and what cues in the image help identify it?" produces a specific answer that can be verified.

As with text prompting, the request should specify what to look at, what level of detail, and what format to respond in. "List every visible product on this shelf, grouped by category, with approximate position (left/centre/right)" is a well-specified prompt that produces structured output. "Tell me about this photo" is a poorly specified prompt that produces paragraph-shaped guesses. The information the model has about the image is the same; what differs is what you are asking it to surface.

24.2 Text-and-image combined

The more powerful pattern is combining text and image in the same prompt. A screenshot of a broken UI alongside a description of the expected behaviour. A chart alongside a question about its data. A photograph of a handwritten page alongside an instruction

to transcribe and correct. The image carries information that would be tedious to describe in words; the text carries the specific request that the image alone would not convey. Prompts that use both modalities naturally are usually much more efficient than prompts that try to describe an image in text before asking the question.

A useful design discipline: if the information you need to share is naturally visual (layout, relative position, colour, handwriting, the look of something), include the image. If you find yourself writing long descriptions of what something looks like in order to ask a text question, you are probably under-using the visual modality. Show, don't tell — the oldest writing advice, newly applicable to prompting.

24.3 Image-out considerations

Prompting models to generate images is a different discipline, with its own conventions that are still rapidly shifting. The patterns that have stabilised: specify the subject clearly, specify the style, specify the composition, and specify what is in the frame and what is not. "A dog" is nearly useless. "A small white terrier, sitting on wet cobblestones in a narrow European alley at night, lit from behind by a streetlamp, shot from low angle with a 35mm lens, shallow depth of field, photorealistic" is a prompt that produces something recognisable.

The details that matter most are the ones that place the image in a specific visual tradition. "Photorealistic" is a tradition. "Watercolour" is a tradition. "1990s anime cel" is a tradition. The model has learned statistical associations with each of these, and naming the tradition causes it to produce an image shaped by those associations. Without a tradition named, you are asking for the average of everything, which produces the characteristic "AI art" look that no serious user wants.

24.4 Audio and video — the frontier

Audio and video prompting are less mature than text and image, but the patterns are emerging quickly. For audio-in tasks (transcription, analysis, question-answering about recordings), the main discipline is to give the model enough context about what the audio is — a conversation, a lecture, a piece of music, an ambient recording — so its analysis is grounded in the right frame. For video, the same discipline applies with the added complication that the model may not attend equally to every frame; specifying what in the video matters ("focus on the interviewee's facial expressions, not the background") improves results.

These modalities are moving fast, and any specific guidance written today will be out of date in six months. The general principle that survives the churn is the one that has survived every generation: structured, specific prompts outperform open-ended ones; context about the modality helps; forbidding unwanted behaviours helps; and the better you can express what you actually want, the better the result.

Multimodal is still prompting

The laws that govern text prompting survive the move to images, audio, and video. Specificity helps. Context helps. Structure helps. The modalities differ in what kinds of information they naturally carry, but the discipline of extracting value from the model is the same. If you have developed good text-prompting habits, you have most of the skill you need for multimodal work already.

Chapter 25 — The Ethics of Prompting

> *"Every capability is a responsibility. The more leverage a tool gives you, the more it matters what you use it for."*
>
> —

A book on prompting that skipped ethics would be a book that treated the subject as merely technical. It is not. Prompting a large language model is a form of speech-at-scale, with consequences that touch people who never agreed to be touched by it. This chapter is a short, honest account of the moral weight of the practice.

25.1 The amplification problem

A model makes every individual prompter dramatically more productive. When the prompter is producing good, true, kind, useful output, this is straightforwardly good. When the prompter is producing spam, propaganda, manipulation, fraud, harassment, or plausible-sounding nonsense that pollutes the information commons, the same multiplier applies. The bad uses are, at the prompt level, indistinguishable from the good ones — a model cannot always tell whether the email it is being asked to draft is a legitimate customer support message or a phishing attack. This is a moral situation for the prompter, not for the model.

A practitioner who takes this seriously audits their own use. The question is not "is this legal" — most of what is possible is legal. The question is "would I defend this use to a room full of strangers who would be affected by the output?" If the answer is no, the technical capability is not a justification. Do not write it. The existence of the tool does not make its misuse acceptable; if anything, the leverage makes the responsibility greater, not less.

25.2 Disclosure and honesty

When model-generated text is passed off as a human's work in contexts where that matters, a trust is broken. Contexts where it matters include: academic submissions when the institution forbids it, creative submissions to venues that require original work, journalism presented as first-hand reporting, personal messages claimed as personal, reviews and testimonials, scientific papers where AI involvement should be declared. The rules vary by community, and the community rules are the right rules to follow. A practitioner's own preferences do not override a community's norms about authorship.

This is not a technology problem. It is an older problem — the problem of honest attribution — in a new form. The honest practice is to know the norms of the context you are writing into and to disclose your tools where the norms require it. In contexts where the norms do not yet exist, the conservative move is to disclose by default. Trust, once broken, is hard to rebuild, and the trust of readers, reviewers, clients, and colleagues is worth more than the short-term convenience of hiding how the work was produced.

25.3 Data, privacy, and consent

When you prompt a model with other people's information — a customer's email, a colleague's draft, a patient's notes — you are making a decision about that information on their behalf. The data may be logged, used for training, reviewed by humans, leaked through a security incident. If the people whose data is in the prompt have not agreed to this, you are spending their trust without asking. The practical discipline: minimise identifiable information in prompts (redact, anonymise, paraphrase); use enterprise or zero-retention offerings for anything sensitive; and never prompt with information whose owner would object if they saw what you were doing.

This is the ethics version of the advice engineers already know about security: the safe default is "don't send". If you need to send,

send the minimum. If you need to send more, make sure you have authority to do so. These habits are not bureaucratic friction; they are how you behave when you take other people's trust seriously, and they are what separates professional from careless use of these tools.

25.4 The ecosystem question

A harder ethical question, with no clean answer, is the effect of model use on ecosystems of human labour. When a model can draft marketing copy, the copywriter's market shrinks. When a model can generate concept art, the concept artist's market shrinks. When a model can produce lesson plans, the teacher's preparation time is, in theory, freed — but in practice, sometimes it is the teacher's job that disappears. Whether the net effect is good or bad depends on policy decisions being made above the level of the individual prompter. But the individual prompter is not absolved by that fact. You can ask, when you use these tools for commercial work, whether you are using them in ways that support or undermine the people whose livelihood is in the same line. There is no universal answer. There is a question that is worth taking seriously.

25.5 A working ethic

A short statement, offered not as a universal moral code but as a starting point for a practitioner's own thinking: use the tools to do more good work, not more work; disclose your tools in contexts where authorship matters; do not send other people's data into systems those people have not agreed to; do not use the amplification to pollute information commons, manipulate vulnerable readers, or replace your own judgement in matters where your judgement is what people are trusting. These are not rules that can be enforced. They are commitments that a person makes to themselves, and the degree to which the practice of

prompting becomes a respectable profession rather than a suspect one will depend on how many individual practitioners make them.

Ethics is not a module to skip

Every time you write a prompt that will produce output other people will read, be affected by, rely on, or pay for, you are acting with leverage that did not exist a decade ago. That leverage has moral weight. Do the work — not merely the technical work of writing good prompts, but the harder work of deciding which prompts are worth writing. A practitioner with that commitment will be trusted; a practitioner without it will, eventually, not be.

Chapter 26 — Where Prompting Is Headed

> *"The craft of prompting will change. The disciplines it teaches — thinking clearly, specifying carefully, communicating precisely — will not."*
>
> —

Any chapter that claims to predict where prompting is headed will be partially wrong within a year and mostly wrong within five. The field is moving too quickly for confident forecasts. But some directions are clear enough to be worth naming, and an operator who reads the directions well will invest their learning time better than one who does not.

26.1 Prompts are becoming documents

Five years ago, a prompt was a sentence. Three years ago, a good prompt was a paragraph. Today, a serious production prompt is often several pages: a system prompt, a set of examples, a library of tools with their own prompts, and a set of conventions that the model is expected to follow across many turns. This trend will continue. Prompts are becoming documents, documents are becoming configurations, and configurations are becoming small programs. The engineer who writes prompts today is doing work that has more in common with building systems than with writing questions.

The implication for the practitioner is that prompt-authorship is a software discipline now. Version control your prompts. Test them. Document them. Review them when they change. Treat them as code, because they behave like code: small changes can cause large effects, regressions happen, and the person who wrote the prompt six months ago may not remember why a specific line is in there. The habits of software engineering — version control, testing, review — apply directly.

26.2 Models are getting better at inferring intent

As models improve, the slack between what you wrote and what you meant narrows. A prompt that would have produced nonsense three years ago often produces reasonable output today, because the model has become better at guessing what you probably wanted. This is mostly good. It means casual users get more value from casual prompts. But it has a subtle cost: practitioners who rely on implicit inference lose the discipline of explicit specification, and their skill ceiling drops. When the task is hard enough to strain the model's inference, the under-disciplined practitioner has no fallback.

The lesson is that the improvement in model inference does not remove the need to learn careful specification; it just hides the need, so that the practitioners who continue to invest in the discipline widen the gap between themselves and those who coast on the model's guesses. If you want to be in the first group, keep the discipline even when the model is being generous. The work you do to specify precisely is work that keeps paying off against harder tasks.

26.3 The interface will dissolve

Today, you prompt a model through a chat interface or an API. Tomorrow, prompting will be more deeply embedded: in your email client, your IDE, your spreadsheet, your browser. The explicit act of writing a prompt will become rarer; the act of shaping a system prompt, choosing defaults, configuring an assistant, will become the primary prompting work for most people. The practitioner's skill will shift from writing one-off prompts to configuring systems that prompt models on behalf of users across many contexts.

This is a higher-leverage form of the same work. A system prompt that is used ten thousand times a day is worth far more attention than a one-off prompt that will be used once. The best practitioners will increasingly do their work at the system-prompt

level — defining the voice, the constraints, the tools, the behaviours — and let the users of those systems prompt in natural language against the scaffolding. If you can do this well, you are building infrastructure that pays dividends on every interaction.

26.4 The skills that will keep mattering

Model capabilities will advance in ways that deprecate specific techniques. What are the durable skills? Clear thinking about what you actually want. Precise specification of outcomes, constraints, and failure modes. Good judgement about when to trust output and when to verify. Literacy in how models work, well enough to diagnose their failures. Writing and editing skill, because the medium is still words. Ethical seriousness about the consequences of what you produce. These were the skills that mattered in 2022, they are the skills that matter in 2026, and everything in the trajectory of the field suggests they will be the skills that matter in 2030.

The specific techniques in this book — few-shot, chain-of-thought, ReAct, self-critique — will be partly superseded by improvements in the models themselves. Some of them will become unnecessary; others will be rediscovered in new forms. That is fine. If you have learned the deeper discipline underneath them — the discipline of asking clearly, structuring carefully, iterating rigorously, and verifying honestly — you will adapt to whatever the models become. The practitioner who has only memorised the tricks will not. Invest in the discipline, and the tricks will take care of themselves.

26.5 A final thought

Prompting is, at its best, a way of thinking more clearly. The act of forcing yourself to specify exactly what you want is a discipline that improves your thinking even when no model is involved. The act of decomposing a problem into parts a model can tackle is a discipline that improves your problem-solving even on problems

you solve yourself. The act of examining output and asking what is missing is a discipline that improves your taste, your rigor, and your standards. If the models disappeared tomorrow, the practitioners who had learned to prompt well would be left with a set of habits that would make them better writers, better thinkers, and better collaborators. The habits are the real prize. The models, for now, are the occasion to develop them.

A closing word

You have reached the end of a book that argued, in many different ways, a single thesis: prompting is not magic, and it is not trivia. It is a craft that rewards clear thinking, honest iteration, and careful specification. The tools will keep changing. The craft will keep mattering. The best thing you can do with what you have learned here is go and use it on something real — a project, a problem, a piece of work that you care about — and in using it, discover what the next chapter should have said. Good prompting to you.

PART

Appendices

• • •

Appendix A — The Prompt Template Library

The templates in this appendix are working scaffolds. They are deliberately generic; fill them in with the specifics of your task, and the underlying structure will carry most of the quality. Each template is annotated with the reasoning behind its structure so that you can adapt it intelligently rather than copy it blindly.

A.1 Analysis and summarisation

> You are [role]. Analyse the following [document type] and produce:\n\n1. A one-sentence summary of the central claim or purpose.\n2. The three to five most important points, in order of importance.\n3. Any claims that appear to be unsupported or require external verification, flagged as [UNVERIFIED].\n4. A short section on what the document does NOT address that a reader might expect it to.\n\nBe concise. Use the author's own framing where possible; do not impose your own categories.\n\n[DOCUMENT]

This template works because it specifies a structured output, explicitly asks for unsupported claims to be flagged, and includes the often-overlooked "what is missing" prompt that surfaces the gaps most readers would not notice on their own.

A.2 Structured extraction

> Extract the following information from the text below and return it as JSON.\n\nSchema:\n{\n "field1": "<description of what should go here; use null if not present>",\n "field2": ["<array of items; empty array if none>"],\n "field3": <number; use null if not determinable>,\n "confidence": "<HIGH | MEDIUM | LOW — your confidence that the extraction is accurate>",\n "notes": "<anything ambiguous or worth flagging>"\n}\n\nRules:\n- Return only valid JSON. No prose before or after.\n- Do not invent information not present in the source.\n- If a field cannot be determined, use null and explain in notes.\n\n[TEXT]

The confidence field and the notes field together reduce the worst failure mode of extraction tasks: confident silent invention of missing data. With them, every extraction has a built-in self-report that lets a downstream reviewer spend effort where effort is needed.

A.3 Editorial critique

> You are a senior editor in [genre/domain]. Critique the draft below. Your job is to identify problems, not to offer praise.\n\nSpecifically, point out:\n- Structural weaknesses (ordering, pacing, missing connective tissue).\n- Clarity problems (ambiguous references, unclear sentences, jargon introduced without definition).\n- Claims that are overreaching, unsupported, or likely to draw pushback.\n- Anywhere the writing sounds generic or could apply to any similar piece without change.\n\nFormat each critique as: PROBLEM: <what is wrong> | LOCATION: <first few words of the relevant sentence> | SUGGESTED FIX: <smallest change that addresses the problem>.\n\n[DRAFT]

The forbidden-praise instruction is essential; without it, models default to the compliment-sandwich structure that buries useful critique in filler. The format requirement produces output that is scannable and actionable, not a wall of prose that the writer has to parse to apply.

A.4 Decision memo

> Produce a decision memo on [question]. Structure:\n\n1. Decision required (one sentence: what is the concrete choice?)\n2. Recommendation (one sentence: what should we do?)\n3. Rationale (three to five bullets: the reasons for the recommendation, in decreasing order of importance)\n4. Key assumptions (two to four bullets: the things that, if wrong, would change the recommendation)\n5. Alternatives considered (for each: the option, the strongest argument for it, why we are not recommending it)\n6. Reversibility (one paragraph: how hard is this to undo if we are wrong, and what would we watch for that would tell us to reverse)\n\nBe specific. Avoid hedging language. If evidence is weak, say so.\n\nContext: [context]

This template front-loads the decision and the recommendation, surfaces assumptions (which are the thing that most often make decisions wrong), and makes reversibility explicit — which shifts the conversation from whether the decision is correct to whether we can afford to find out.

A.5 Learning — explain with checkpoints

> Explain [topic] to me at my level ([level description: e.g., 'undergraduate with one course in statistics']).\n\nAfter each major section, pause and ask me a short question that checks whether I have understood it. Do not continue to the next section until I answer. If my answer is wrong, do not give me the correct answer — ask a smaller question that helps me find it myself.\n\nBegin by asking me what specific aspect of [topic] I am most interested in understanding, and tailor the explanation around that.

This template activates three separate learning mechanisms: level-appropriate explanation, active retrieval (the questions), and Socratic correction (no direct answers to wrong responses). It is slower than a normal explanation, which is precisely why it produces better learning.

A.6 Code — implementation with tests first

> I need a function with this interface:\n\n[signature and type definitions]\n\nBehaviour:\n- [describe each intended behaviour]\n- [describe edge cases]\n- [describe failure modes]\n\nStep 1: Write a test file that exercises every behaviour, edge case, and failure mode listed above. Use [testing framework]. Do not write the implementation yet.\n\nI will review the tests. Once I approve them, write the implementation. The implementation should pass every test and should not modify the test file.

Test-first prompting is slower in the short run and dramatically faster overall, because reviewing tests is easier than reviewing implementations and the resulting code is more reliable. The explicit "do not modify the test file" line is needed because agents sometimes fix failing tests by changing the test rather than the code.

A.7 Debugging — hypothesise before fixing

> I am debugging a problem. Before proposing a fix, help me rank hypotheses.\n\nExpected behaviour: [what should happen]\nActual behaviour: [what is happening]\nWhat I have tried: [list]\nRelevant code: [paste]\nError / logs: [paste in full, including stack trace]\n\nList the five most likely root causes, ranked by probability. For each, include:\n- The evidence that supports this hypothesis.\n- The evidence that is against it.\n- A quick diagnostic step that would confirm or rule it out.\n\nDo not propose a fix yet. We will discuss the hypotheses first.

This template is the antidote to the most common debugging failure mode: a confident fix applied to the wrong cause, which masks the real bug and makes it harder to find later.

A.8 Marketing — hero copy with banned words

> Write a landing-page hero section for [product].\n\nTarget reader:\n[one paragraph: who they are, what they are trying to do, what is frustrating them, what they have already tried]\n\nWhat would make them stop scrolling:\n[one paragraph: the specific promise that would be surprising and credible to this reader]\n\nLength: 100–140 words.\n\nBanned

> words: empower, unlock, transform, revolutionise, leverage, seamless, cutting-edge, solution, streamline, innovative, world-class, best-in-class, next-generation, game-changing.\n\nWrite three variations with meaningfully different angles (not just word swaps). Label each variation with the specific promise it is leading with.

The banned-words list does more work than any other line in this prompt. It forces the model out of its default marketing-register into something more specific. The three-variation structure gives you options to pick from and surfaces which angle is worth pursuing further.

A.9 Research — steelman and stress-test

> Consider the claim: [claim].\n\nStep 1: Build the strongest possible case FOR this claim. Assume an intelligent reader who is skeptical. Use real evidence where you can; flag where you are reasoning from general principles rather than specific evidence.\n\nStep 2: Build the strongest possible case AGAINST this claim, with the same standards.\n\nStep 3: Identify where the two cases actually disagree (point to specific claims or pieces of evidence), versus where they are talking past each other (different frames, definitions, or priorities).\n\nStep 4: Propose the single most informative piece of evidence that, if we had it, would most reduce our uncertainty about the claim.

This template is unusually productive for contested questions because it forces the analysis to treat both sides seriously, identifies the actual point of disagreement (often much narrower than the apparent disagreement), and converts the debate into a research question that can, at least in principle, be resolved.

A.10 System prompt — assistant persona

> You are [name], a [role] for [product/user type].\n\nYour job is to [primary purpose in one sentence].\n\nVoice: [three adjectives]. Do not be [two or three adjectives to avoid].\n\nYou ALWAYS:\n- [behaviour 1]\n- [behaviour 2]\n- [behaviour 3]\n\nYou NEVER:\n- [forbidden behaviour 1]\n- [forbidden behaviour 2]\n- [forbidden behaviour 3]\n\nWhen you don't know something, say 'I don't know' rather than guessing. When a user asks for something outside your role (such as [examples]), politely redirect them to [alternative].\n\nFormat: [default output format — tone, length, structure].\n\nKnowledge scope: [what topics you are expected to know about; what you should deflect].

This template is the skeleton of a system prompt. The concrete content of each bracketed section is where the work happens, but the skeleton ensures nothing important is forgotten. The

'ALWAYS' and 'NEVER' lists, in particular, are worth investing in — they are the most direct way to pin down behaviour that would otherwise drift.

Appendix B — Common Pitfalls and How to Avoid Them

Every common failure mode in prompting has a name and a fix. This appendix is a catalogue. Read it once carefully; then keep it bookmarked for when the symptoms show up.

B.1 The generic-output pitfall

Symptom: the output is fluent but could have been written for any reader about any product. It uses business-register vocabulary, sandwich-structured praise, and makes no specific claims. Cause: the prompt was underspecified, so the model reverted to the statistical centre of its training. Fix: add a target reader, forbid filler words, require specific claims, and where possible, include examples of the quality level you want.

B.2 The confident-hallucination pitfall

Symptom: the model produces specific facts — numbers, names, citations, URLs — that sound authoritative and are wrong. Cause: the fluency of the prose triggers the reader's trust, while the underlying mechanism is plausibility, not truth. Fix: require sources for every claim, require arithmetic to be shown, mark unverified claims explicitly, and for high-stakes work, route the output through a verification pass.

B.3 The lost-in-the-middle pitfall

Symptom: the model ignores or under-weights instructions that appeared in the middle of a long prompt. Cause: models attend more strongly to the beginning and end of their context. Fix: put the most important instructions at the end of the prompt (closest to where generation begins) or repeat them. Never bury a critical constraint in the middle of a long document.

B.4 The over-helpful pitfall

Symptom: the model adds content you did not ask for — preambles, summaries of what it is about to do, disclaimers, helpful notes. Cause: the model has learned from training that helpful elaboration is often welcomed. Fix: explicitly forbid the specific forms you do not want. "Respond only with the requested JSON. No prose before or after. No explanation of your reasoning."

B.5 The tone-drift pitfall

Symptom: over a long conversation or long output, the model's tone gradually shifts toward its default voice, regardless of the voice you originally specified. Cause: long-range adherence to tone instructions is imperfect; each generation is pulled slightly toward the training average. Fix: restate the tone instruction periodically (in a long conversation) or at the top of each section of a long output. For production systems, put tone instructions in the system prompt, not just the initial message.

B.6 The premature-solution pitfall

Symptom: the model offers a specific solution before you have fully specified the problem, and the solution solves the wrong version of the problem. Cause: models are trained to be helpful quickly; they will answer as soon as they have enough context to produce something. Fix: explicitly ask for clarifying questions first. "Before you propose a solution, ask me any questions you need answered to make a good recommendation. I will answer, and then you will propose."

B.7 The over-fit-to-example pitfall

Symptom: when you provided few-shot examples, the model copies surface features of the examples (their length, their specific vocabulary, their exact structure) rather than the underlying pattern. Cause: examples that are too similar to each other teach the model the wrong generalisation. Fix: choose diverse examples

that differ in surface features but share the underlying pattern you want. Three well-chosen diverse examples outperform ten near-identical ones.

B.8 The instruction-contradiction pitfall

Symptom: the output follows some of your instructions and ignores others, in a way that looks random but is actually driven by conflicts you did not notice. Cause: two or more instructions in the prompt are in tension, and the model chooses one, often the most recent or the most specific. Fix: audit long prompts for contradictions. Common offenders: asking for both brevity and completeness; asking for both formality and casualness; asking for structured output while also saying "write naturally"; asking for creativity while also forbidding departures from a template.

B.9 The over-delegation pitfall

Symptom: you asked the model to make a decision that required judgement or values beyond the information in the prompt, and the decision it made is bad. Cause: you let the model decide something you should have decided. Fix: keep judgement calls, values trade-offs, and stakeholder-sensitive decisions in your own hands. Use the model to execute the decision, not to make it.

B.10 The iteration-overfitting pitfall

Symptom: a prompt that worked well on your test cases is worse on new inputs. Cause: you iterated your prompt against a small sample, and the prompt became specifically tuned to that sample's quirks. Fix: test prompt changes against a wider distribution before declaring them improvements. If you are iterating aggressively, hold out a test set that you do not iterate against, and re-check performance on it periodically.

Appendix C — Glossary

A short reference of terms used throughout the book. Definitions here are operational — what you need to know to use the term correctly in practice — not academic.

Agent

A configuration in which the model can take actions (call tools, read and write files, execute code) in a loop, observing the results of each action and deciding what to do next, until a goal is achieved or abandoned.

Chain-of-Thought (CoT)

A prompting technique in which the model is asked to show its reasoning step-by-step before producing a final answer. Substantially improves performance on problems that require multi-step reasoning. See Chapter 5.

Context window

The maximum number of tokens a model can consider at once, including both the prompt and its generated output. Modern frontier models have context windows in the hundreds of thousands of tokens, but model attention within the window is uneven, with stronger attention at the start and end than in the middle.

Few-shot prompting

A prompting technique in which the model is given a small number of input-output examples (typically 2–10) demonstrating the desired behaviour, before being asked to produce output for a new input. See Chapter 4.

Hallucination

Model output that is fluent and plausible-sounding but factually incorrect. Not a moral failure by the model; a structural consequence of the plausibility-based next-token prediction mechanism. See Chapter 17.

Meta-prompt

A prompt used to generate or refine another prompt. Useful for bootstrapping templates, for rewriting prompts that are underperforming, and for producing specialised prompts from a general description. See Chapter 13.

Prompt

The input given to a language model, including instructions, context, examples, and the specific task. Modern prompts often include a system prompt (setting persistent behaviour) and one or more user messages.

ReAct

A prompting pattern combining Reasoning and Acting, in which the model alternates between producing thoughts (internal reasoning) and actions (calls to tools or the environment), observing results, and iterating. The foundation of most modern agentic systems. See Chapter 10.

RAG (Retrieval-Augmented Generation)

A system in which a retrieval step (often a vector search over a document corpus) is performed before generation, and the retrieved material is inserted into the prompt to ground the model's output. See Chapter 12.

Self-consistency

A technique in which the same prompt is sampled multiple times and the most common answer is taken as the final output.

Improves reliability on problems with a single correct answer at the cost of more computation. See Chapter 8.

System prompt

A prompt intended to set persistent behaviour for the model across a conversation or a deployment. Typically contains role, voice, constraints, and forbidden behaviours. Distinguished from user messages by position and, in some APIs, by an explicit role field.

Temperature

A sampling parameter controlling how much randomness the model uses when choosing between plausible next tokens. Lower temperatures produce more deterministic, conservative output; higher temperatures produce more varied, creative output. For most analytical and factual tasks, low temperature (0–0.3) is preferred.

Token

The unit of text the model processes. Tokens are fragments of words, usually 3–4 characters on average in English. Model limits, pricing, and context windows are all expressed in tokens.

Tree of Thoughts (T.O.T)

A technique in which the model explores multiple branches of reasoning in parallel, evaluates them, and selects or combines the best path. Useful for problems with many possible approaches and uncertain solution paths. See Chapter 9.

Zero-shot prompting

A prompting technique in which the model is given only an instruction, with no examples. The baseline against which few-shot is usually compared. See Chapter 4.

About The Author:

Veer Ashish Sukhadiya

Author | Children's Literature | Motivational Writing | Fiction Writing

Veer Ashish Sukhadiya is an Indian author known for writing children's books, fiction, and motivational content for students and young readers. His work focuses on encouraging curiosity, positive thinking, moral values, and consistent self-growth among children and teenagers. Through simple language and relatable themes, he aims to inspire young minds to read, learn, and develop confidence for life beyond academics.

As an author, Veer Sukhadiya believes that books can shape a child's thinking and character. His writing style combines storytelling with meaningful messages, making learning enjoyable and impactful. His books are written with the intention of supporting students, parents, and educators in nurturing strong values and a growth-oriented mindset.

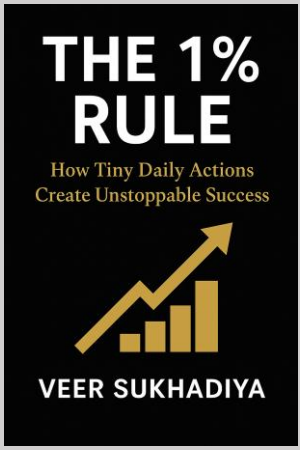
Colophon

First edition. June 2026.

From this Author

"How Tiny Daily Actions
Create Unstoppable Success"

A Motivational Book to find you your 1% Rule



The 1% Rule

Available on our Official Store veeerbooks.in

Format: Ebook and Paperback.

For More Contact-
veeersukhadiyabooks95@gmail.com

From this Author

"A Broken Kingdom. A Returning Fire."

When Truth Refuses to Burn

The Circle of Ash

Available on our Official Store veeerbooks.in

Format: Ebook and Paperback.

For More Contact-
veeersukhadiyabooks95@gmail.com